ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN

KỸ THUẬT DỮ LIỆU (CO5173)

Data Engineering

Đề tài DE_261

NỀN TẢNG KỸ THUẬT DỮ LIỆU CHO HỆ THỐNG GIÁO DỤC TRỰC TUYẾN QUY MÔ LỚN

Learning Analytics, gợi ý học tập và cảnh báo sớm nguy cơ bỏ học

---

Giảng viên hướng dẫn	PGS. TS. Võ Thị Ngọc Châu
Lớp / Nhóm	[Lớp] / [Nhóm]
Học kỳ	Học kỳ 1, năm học 2026–2027

Học viên thực hiện

Họ và tên	**MSHV**	**Phần việc chính**
[Phạm Duy Anh]	[2310139]	[BR4, BR5]
[Họ và tên thành viên 2]	[MSHV]	[Phần việc]
[Họ và tên thành viên 3]	[MSHV]	[Phần việc]
[Họ và tên thành viên 4]	[MSHV]	[Phần việc]

TP. Hồ Chí Minh, tháng 11 năm 2026

Mục lục

Tóm tắt **6**

Danh mục thuật ngữ và từ viết tắt **7**

1 Giới thiệu **9**

1.1 Bối cảnh . . . 9
1.2 Phát biểu bài toán . . . 9
1.3 Mục tiêu và phạm vi . . . 9
1.4 Đóng góp chính . . . 10
1.5 Phương pháp thực hiện . . . 10
1.6 Cấu trúc báo cáo . . . 11

2 Phân tích yêu cầu và nguồn dữ liệu **11**

2.1 Các bên liên quan và yêu cầu nghiệp vụ . . . 11
2.2 Nguồn dữ liệu . . . 12
2.3 Yêu cầu phi chức năng . . . 13
2.4 Ánh xạ yêu cầu vào pipeline . . . 14

3 Kiến trúc, công nghệ và kho dữ liệu **15**

3.1 Kiến trúc giải pháp . . . 15
3.1.1 Tổng quan kiến trúc . . . 15
3.1.2 Lựa chọn công nghệ . . . 16
3.1.3 Kiến trúc phần mềm: DDD và kiến trúc lục giác . . . 18
3.1.4 Ngữ nghĩa xử lý luồng . . . 18
3.1.5 Các quyết định kiến trúc . . . 19
3.1.6 Điều chỉnh so với kiến trúc tham chiếu của đề bài . . . 20
3.2 Lakehouse và kho dữ liệu . . . 21
3.2.1 Kiến trúc medallion . . . 21
3.2.2 Chuẩn hoá telemetry và tiến hoá schema . . . 22
3.2.3 Khử định danh và phân quyền . . . 22

3.2.4 Mô hình chiều Kimball ở lớp gold 23
3.2.5 Data mart theo yêu cầu nghiệp vụ 25
3.2.6 Thiết kế vật lý 26
3.2.7 Cổng kiểm tra chất lượng dữ liệu 26
3.2.8 Điều phối, dữ liệu trễ và chạy lại 27

4 Hiện thực và giao diện ứng dụng 29
4.1 Thiết kế và hiện thực theo từng yêu cầu 29
4.1.1 BR1 — Pipeline thông báo hướng sự kiện 29
4.1.2 BR2 — Gợi ý khoá học và bước học tiếp theo 30
4.1.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết 31
4.1.4 BR4 — Phát hiện câu hỏi và test case bất thường 31
4.1.5 BR5 — Cảnh báo sớm nguy cơ bỏ học 33
4.1.6 BR6 — Tiến độ kỹ năng theo tổ chức 35
4.1.7 BR7 — Heatmap tương tác video 35
4.1.8 BR8 — Đồng bộ nội dung và xu hướng tương tác 35
4.2 Giao diện người dùng 37
4.2.1 Quy trình thiết kế 37
4.2.2 Hệ thống thiết kế 37
4.2.3 Kiến trúc frontend 38
4.2.4 Xác thực và phạm vi dữ liệu 38
4.2.5 Các màn hình chính 39
4.2.6 Kiểm thử giao diện 40

5 Kiểm thử và đánh giá 52
5.1 Chiến lược kiểm thử 52
5.2 Truy vết yêu cầu và kết quả sử dụng 52
5.3 Môi trường và phương pháp đo 54
5.4 Kết quả hiệu năng 54
5.4.1 BR4: PostgreSQL SQL và Spark SQL 54
5.4.2 BR4: sửa điểm qua worker triển khai 56

5.4.3 BR5: Spark Structured Streaming và Flink 56
5.4.4 BR6–BR8: phục vụ mart 58
5.4.5 BR8: từ xuất bản đến khi tìm kiếm được 59
5.4.6 BR7/BR8: lớp phủ thời gian thực 59
5.4.7 Thời gian build kho dữ liệu 60
5.5 Hiệu quả cảnh báo BR5 trên OULAD 60
5.6 Thảo luận 61

6 Hạn chế, hướng phát triển và kết luận 61
6.1 Hạn chế và hướng phát triển 61
6.1.1 Hạn chế hiện tại 61
6.1.2 Hướng phát triển 62
6.2 Kết luận 63

Tài liệu tham khảo 64

A Hướng dẫn chạy và tái lập kết quả 66
A.1 Yêu cầu 66
A.2 Khởi động 66
A.3 Kiểm thử 66
A.4 Đo hiệu năng 67
A.5 Biên dịch báo cáo 67

B Tóm tắt API 67

Danh sách hình vẽ

1 Kiến trúc tổng thể của nền tảng DE_261 16
2 Các lớp của lakehouse và tần suất cập nhật 21
3 Trích tập luật thông báo (`infra/notification/rules.yaml`) 30
4 Từ CDC đến kết quả phục vụ BR4 và BR5 32
5 Các cửa sổ đặc trưng tại mốc đánh giá T (BR5) 33
6 Đồng bộ nội dung BR8 dựa trên CDC 36
7 Form đăng nhập theo vai trò trước khi nhập mật khẩu 39
8 Learner Home trong Classroom. Người học Chi thấy khoá học, thông báo và tiến độ bài học 40
9 Code Lab trong Classroom. Lời giải FizzBuzz nhận phản hồi đã chấm và kết quả theo test 41
10 Assessment Health trong Persistent với fixture Python for Data 42
11 Vòng đời cảnh báo trong Learners to support, chế độ Persistent 43
12 Assessment Health trong Replay, có nhãn Simulated data 44
13 Video Insights trong Persistent. Mart Trino hiển thị phiên bản 2 của video Joins đến 17/10/2026 45
14 Engagement trong Persistent. Mart Trino hiển thị xu hướng theo tuần của SQL Foundations 46
15 Content & Search trong Persistent. Transcript Joins đã đồng bộ và có thể tìm kiếm 47
16 Organisation Progress trong Persistent với snapshot kho ngày 17/10/2026 . . . 48
17 Phiên Admin chỉ có phạm vi Acme. Yêu cầu kiểm chứng đến API Globex trả 403 49
18 Pipeline Health. Các bộ đếm và topic của luồng demo 50
19 Learner Home trong Classroom ở chiều rộng 390 px 51
20 BR4, tổng hợp và ghi mart qua harness 55
21 BR5, Kafka đến cửa sổ đã lưu 57
22 Độ trễ phục vụ các mart BR6–BR8 (30 request mỗi trường hợp) 58
23 BR8: độ trễ từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp 59
24 Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần) . . 60

Danh sách bảng

2 Yêu cầu nghiệp vụ và nhu cầu kỹ thuật dữ liệu . . . 11
3 Bốn nguồn dữ liệu và cách tiếp nhận . . . 12
4 Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime . . . 14
5 Công nghệ theo vai trò . . . 17
6 Các lớp trong một bounded context . . . 18
7 Tóm tắt các quyết định kiến trúc (ADR) . . . 19
8 Hợp đồng dữ liệu của từng lớp . . . 21
9 Phân quyền truy cập kho dữ liệu qua Trino . . . 23
10 Ma trận bus của lớp gold (D: vai trò ngày, •: khoá chiều) . . . 24
11 Các data mart và kho đặc trưng . . . 25
12 Các cổng kiểm tra chất lượng . . . 27
13 Các DAG Airflow . . . 28
14 Cơ chế xử lý dữ liệu trễ, trùng lặp và sai thứ tự . . . 28
15 Các tầng của luật gợi ý (BR2) . . . 31
16 Quy tắc phát hiện bất thường của BR4 (cấu hình được) . . . 33
17 Các tín hiệu rủi ro và điểm (BR5) . . . 34
18 Trọng số hoạt động trong điểm tương tác (`EngagementWeights`) . . . 36
19 Workspace, đường dẫn và trang . . . 38
20 Các tầng kiểm thử . . . 52
21 Từ yêu cầu và dữ liệu đến kết quả, minh chứng và giới hạn . . . 53
22 BR4, xử lý và ghi mart qua harness (ms) . . . 55
23 BR4, sửa điểm qua CDC với harness điều phối (ms) . . . 55
24 BR4 triển khai, sửa điểm từ 2/24 lên 3/24 (giây) . . . 56
25 BR5, Kafka đến cửa sổ đã lưu (ms) . . . 57
26 BR5, commit nguồn đến API Advisor trong benchmark (ms) . . . 57
27 Độ trễ từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước) . . . 59
28 BR5 trên OULAD, gộp ba mốc quan sát . . . 61
29 Các endpoint chính . . . 68

Tóm tắt

Báo cáo trình bày việc thiết kế và hiện thực **DE_261**, một nền tảng kỹ thuật dữ liệu cho hệ thống giáo dục trực tuyến quy mô lớn. Nền tảng nhận bốn nguồn dữ liệu rời rạc: luồng clickstream telemetry từ web và mobile, nhật ký thay đổi giao dịch (CDC) từ PostgreSQL, kho tài liệu học tập phi cấu trúc và nhật ký thực thi bài nộp từ sandbox. Từ đó, nền tảng biến chúng thành các kho dữ liệu phục vụ tám yêu cầu nghiệp vụ (BR1–BR8) của bốn nhóm người dùng: người học, giảng viên, cố vấn học tập và quản trị viên tổ chức.

Kiến trúc dùng Apache Kafka và Debezium làm tầng tiếp nhận và event bus. Apache Spark (batch và Structured Streaming) là engine xử lý chính. Delta Lake trên object storage (MinIO) tổ chức theo mô hình medallion bronze–silver–gold, với lớp gold là một kho dữ liệu dạng lược đồ hình sao Kimball. Kho được Hive Metastore lập danh mục và Trino phục vụ truy vấn SQL. Redis là tầng phục vụ độ trễ thấp, OpenSearch phục vụ tìm kiếm lai (keyword và vector). Mọi pipeline được tổ chức thành bảy *bounded context* theo Domain-Driven Design với kiến trúc lục giác (hexagonal), nên cùng một use case chạy được trên nhiều engine và kiểm thử được mà không cần engine. Mười bốn quyết định kiến trúc được ghi lại bằng ADR, bao gồm các điểm nhóm chủ động điều chỉnh so với kiến trúc gợi ý của đề bài.

Về kết quả đo trên máy cá nhân với dữ liệu tổng hợp:

- **BR4**: sửa một kết quả chấm qua CDC đổi tỷ lệ đạt từ $2/24$ thành $3/24$. Spark SQL/Delta công bố bản mới theo phiên bản nguồn. PostgreSQL SQL là phương án đối chiếu.
- **BR5**: luồng CDC đến cảnh báo mất 20,97 giây trong một lượt nghiệm thu. Sau khi khởi động lại Spark, hoạt động mới giảm điểm từ 100 xuống 50 mà không sinh cảnh báo trùng. Phép đo đối chiếu Spark/Flink và đánh giá OULAD được trình bày ở Mục 5.
- **BR8**: nội dung khoá học đã xuất bản trở nên tìm kiếm được sau trung vị 535 ms.
- **BR7, BR8**: lớp phủ thời gian thực hiển thị sau khoảng 7 giây, ổn định khi tải tăng 15 lần.
- **BR6–BR8**: các data mart được đọc thẳng qua Trino với trung vị 42–92 ms, và qua API có cache với trung vị 1,4–2,5 ms.
- **Kho dữ liệu**: toàn bộ kho (43 model) được xây trong khoảng 2–2,5 phút và vượt qua mọi cổng kiểm tra chất lượng.

Đợt hồi quy sau cập nhật xác thực có 379 bài kiểm thử Python/PostgreSQL đạt. Giao diện cung cấp phiên đăng nhập riêng cho giảng viên, cố vấn và quản trị viên. Báo cáo phân biệt các lượt đo trên hệ thống chạy thật, các kịch bản mô phỏng và đánh giá ngoại tuyến.

Từ khoá: kỹ thuật dữ liệu, lakehouse, Delta Lake, Kafka, Debezium, CDC, Spark Structured Streaming, kho dữ liệu Kimball, learning analytics, cảnh báo bỏ học.

Danh mục thuật ngữ và từ viết tắt

Thuật ngữ	Giải thích
ADR	*Architecture Decision Record*: tài liệu ghi lại một quyết định kiến trúc, bối cảnh, phương án bị loại và hệ quả.
BR	*Business Requirement*: yêu cầu nghiệp vụ (BR1–BR8) trong đề bài.
Bronze / Silver / Gold	Ba lớp của kiến trúc medallion: dữ liệu thô, dữ liệu đã làm sạch và khử định danh, dữ liệu mô hình hoá cho phân tích.
CDC	*Change Data Capture*: bắt các thay đổi cấp dòng từ nhật ký giao dịch (WAL) của cơ sở dữ liệu.
CDF	*Change Data Feed*: tính năng của Delta Lake ghi lại các thay đổi cấp dòng của bảng.
DDD	*Domain-Driven Design*: thiết kế hướng miền nghiệp vụ, chia hệ thống thành các *bounded context*.
DLQ	*Dead-Letter Queue*: nơi lưu sự kiện bị từ chối kèm lý do để xử lý lại.
Event time	Thời điểm sự kiện xảy ra tại nguồn, khác với thời điểm hệ thống nhận được.
Exactly-once	Đảm bảo mỗi bản ghi tác động lên kết quả đúng một lần dù có lỗi hoặc gửi lại.
Hexagonal	Kiến trúc lục giác (ports and adapters): lõi nghiệp vụ chỉ phụ thuộc vào các cổng trừu tượng.
HMS	Hive Metastore: dịch vụ danh mục (catalog) chung cho các bảng Delta.
Idempotent	Thực thi lặp lại nhiều lần cho cùng kết quả như thực thi một lần.
k-anonymity	Nguyên tắc không công bố số liệu của nhóm có ít hơn k cá thể; ở đây $k = 5$.
Lease	Quyền có thời hạn để worker xử lý một job. Số thế hệ lease tăng khi worker nhận lại job.
LSN	*Log Sequence Number*: vị trí trong WAL của PostgreSQL, dùng để sắp thứ tự thay đổi.
Mart	Bảng tổng hợp phục vụ một yêu cầu nghiệp vụ cụ thể.
Medallion	Mô hình tổ chức lakehouse theo các lớp chất lượng tăng dần.
OLTP	*Online Transaction Processing*: cơ sở dữ liệu giao dịch của hệ thống quản lý học tập (LMS).

Thuật ngữ	Giải thích
Outbox	Bảng ghi sự kiện cần gửi, được ghi cùng giao dịch với dữ liệu nghiệp vụ.
PII	*Personally Identifiable Information*: thông tin định danh cá nhân.
Publication	Một lần công bố kết quả BR4 với định danh riêng, gắn với revision nguồn và engine.
Pseudonym	Bút danh: giá trị băm có khoá (HMAC) thay cho định danh thật.
SCD	*Slowly Changing Dimension*: chiến lược lưu lịch sử thay đổi của bảng chiều (loại 1 ghi đè, loại 2 lưu phiên bản).
Source revision	Dấu vết của đầu vào BR4 đã chuẩn hóa, dùng để xác định mart có bao phủ thay đổi mới hay chưa.
Watermark	Ngưỡng thời gian sự kiện mà engine streaming coi là đã nhận đủ dữ liệu trước đó.

1 Giới thiệu

1.1 Bối cảnh

Các nền tảng giáo dục trực tuyến quy mô lớn sinh ra dữ liệu liên tục và đa dạng. Mỗi lần người học mở bài giảng, tua video, nộp bài lập trình hay thảo luận đều để lại dấu vết. Hệ thống quản lý học tập (LMS) ghi nhận tiến độ, bài nộp và điểm số. Kho học liệu gồm transcript video, slide và tài liệu PDF được cập nhật theo từng phiên bản. Các sandbox chấm bài sinh ra nhật ký thực thi chi tiết. Những dữ liệu này có giá trị lớn cho *learning analytics*: giúp giảng viên biết bài tập nào có vấn đề, giúp cố vấn học tập phát hiện sớm người học có nguy cơ bỏ dở, và giúp người học nhận phản hồi và gợi ý kịp thời.

Tuy vậy, các nguồn này rời rạc, khác nhau về định dạng, tốc độ và độ tin cậy. Telemetry từ thiết bị di động có thể đến trễ nhiều ngày hoặc sai thứ tự. Thay đổi trong cơ sở dữ liệu giao dịch phải được bắt theo đúng thứ tự mà không làm nặng hệ thống production. Tài liệu học tập cần được chia đoạn và quản lý phiên bản. Dữ liệu cá nhân của người học cần được bảo vệ, nhất là khi báo cáo cho các tổ chức đối tác (B2B). Đây là bài toán kỹ thuật dữ liệu điển hình, đòi hỏi kết hợp xử lý luồng, xử lý theo lô, lưu trữ phân tích và phục vụ độ trễ thấp [1].

1.2 Phát biểu bài toán

Đề bài DE_261 yêu cầu xây dựng một nền tảng dữ liệu biến bốn nguồn dữ liệu thô thành các kho dữ liệu phục vụ ba mục tiêu: phân tích học tập, hệ thống gợi ý thích ứng và phát hiện sớm nguy cơ bỏ học. Đề bài xác định tám yêu cầu nghiệp vụ (BR1–BR8) cho bốn nhóm người dùng (người học, giảng viên, cố vấn học tập, quản trị viên tổ chức). Đề bài cũng gợi ý một kiến trúc tham chiếu gồm Kafka và Debezium cho tiếp nhận, Flink cho xử lý luồng, Spark cho xử lý lô, Delta Lake hoặc Iceberg cho lakehouse, và Redis hoặc Cassandra cho tầng phục vụ. Mục 2 phân tích chi tiết các yêu cầu này.

1.3 Mục tiêu và phạm vi

Nhóm đặt ra các mục tiêu sau:

1. Thiết kế một kiến trúc dữ liệu đầu cuối bao phủ cả tám yêu cầu nghiệp vụ, bám sát kiến trúc tham chiếu. Mọi điểm điều chỉnh đều có lý do và được ghi lại.
2. Hiện thực các pipeline chạy được trên máy cá nhân bằng Docker Compose: tiếp nhận CDC thật qua Debezium, lakehouse Delta Lake với kho dữ liệu dạng hình sao, các data mart theo từng BR, và các đường xử lý thời gian thực.

3. Bảo đảm các tính chất quan trọng của hệ thống dữ liệu: xử lý theo thời gian sự kiện, chịu được dữ liệu trễ và trùng lặp, khả năng chạy lại idempotent, khử định danh và k-anonymity, kiểm tra chất lượng dữ liệu.
4. Cung cấp API và giao diện web cho từng nhóm người dùng, để kết quả của pipeline thực sự được sử dụng.
5. Đánh giá tính đúng đắn và hiệu năng bằng kiểm thử tự động và đo đạc có thể tái lập.

Các lượt chạy ứng dụng và đo hiệu năng dùng dữ liệu tổng hợp. Đánh giá hồi cứu policy BR5 dùng dữ liệu học tập đã ẩn danh của Open University [2]. Hệ thống được triển khai trên một máy. Kết quả hiệu năng chỉ áp dụng cho cấu hình cục bộ được ghi trong nhật ký.

1.4 Đóng góp chính

- Một **lakehouse Delta Lake** ba lớp với 50 bảng và 43 model SQL. Lớp gold là kho dữ liệu Kimball gồm 9 bảng chiều và 11 bảng sự kiện, dùng khoá thay thế tất định. Các mart cho BR4, BR6, BR7, BR8 và một kho đặc trưng ngoại tuyến cho BR2. Kho được lập danh mục bằng Hive Metastore và phục vụ qua Trino có phân quyền theo vai trò.
- Một đường **CDC thật**: PostgreSQL WAL, rồi Debezium, Kafka, Spark SQL/Delta đến mart đánh giá BR4 và Spark Structured Streaming đến cửa sổ, điểm rủi ro và cảnh báo BR5. Đường này chịu được phát lại (replay), cập nhật và xoá, và khởi động lại checkpoint.
- Đồng bộ nội dung **BR8 dựa trên CDC**: vô hiệu hoá cache Redis, chia đoạn và nhúng vector vào OpenSearch, tìm kiếm lai, với ngữ nghĩa idempotent theo phiên bản.
- **Lớp phủ thời gian thực** cho heatmap video (BR7) và điểm tương tác (BR8) trên Redis, ghép với mart theo ngày.
- Mười bốn **ADR**, một kiến trúc phần mềm DDD/hexagonal có test bảo vệ ranh giới, và một bộ kiểm thử nhiều tầng có kèm bằng chứng đo đạc.
- Một **giao diện web** mới cho năm không gian làm việc (workspace), thiết kế từ mockup và nối với API thật.

1.5 Phương pháp thực hiện

Nhóm làm việc theo vòng lặp ngắn: phân tích yêu cầu, ghi quyết định kiến trúc (ADR), hiện thực, rồi kiểm chứng bằng test và số đo. Lõi nghiệp vụ (các *policy* và *use case*) được viết trước, kiểm thử độc lập với engine. Sau đó các adapter cho Kafka, Spark, Delta, Redis, PostgreSQL và OpenSearch được bổ sung dần. Mỗi nhóm BR có một tài liệu kế hoạch và một tài liệu hiện thực trong thư mục `docs/`. Các tài liệu này ghi rõ cái gì đã chạy thật, cái gì còn là demo, và các lỗi

tìm ra trong quá trình kiểm chứng.

1.6 Cấu trúc báo cáo

- Mục 2 phân tích người dùng, yêu cầu nghiệp vụ, nguồn dữ liệu và yêu cầu phi chức năng.
- Mục 3 trình bày kiến trúc, lựa chọn công nghệ, lakehouse và kho dữ liệu.
- Mục 4 mô tả cách hiện thực BR1–BR8 và cách người dùng khai thác kết quả trên giao diện.
- Mục 5 trình bày tính đúng đắn, hiệu năng và hiệu quả khai thác dữ liệu.
- Mục 6 tổng kết kết quả, các hạn chế và hướng phát triển.

2 Phân tích yêu cầu và nguồn dữ liệu

2.1 Các bên liên quan và yêu cầu nghiệp vụ

Đề tài DE_261 xác định bốn nhóm người dùng và tám yêu cầu nghiệp vụ. Theo CO5173, tám BR đạt mức tối thiểu $2n$ nếu nhóm có bốn thành viên. Thông tin nhóm sẽ được xác nhận trên bìa. Bảng 2 tóm tắt mỗi yêu cầu cùng nhu cầu kỹ thuật dữ liệu tương ứng, theo cách đọc của nhóm.

Bảng 2. Yêu cầu nghiệp vụ và nhu cầu kỹ thuật dữ liệu

Mã	Người dùng	Yêu cầu nghiệp vụ	Nhu cầu kỹ thuật dữ liệu
BR1	Người học	Nhận thông báo khi có bài tập mới, deadline sắp đến hoặc nội dung được cập nhật	Pipeline thông báo hướng sự kiện: nhận sự kiện LMS qua message broker, lọc bằng rule engine, gửi theo thời gian thực
BR2	Người học	Nhận gợi ý khoá học và bước học tiếp theo	Pipeline đặc trưng hai nhánh: đặc trưng theo lô (Spark) và đặc trưng phiên (streaming) qua feature store
BR3	Người học	Nhận phản hồi chi tiết khi nộp bài lập trình/SQL	Tiếp nhận bài nộp tách rời qua hàng đợi đệm; chuẩn hoá kết quả kiểm thử (stderr, diff test case, thời gian chạy)
BR4	Giảng viên	Phát hiện câu hỏi hoặc test case có tỷ lệ trượt bất thường	Aggregation mart: tỷ lệ đạt theo từng test case và câu hỏi để lọc đề lỗi logic hoặc quá khó

Mã	Người dùng	Yêu cầu nghiệp vụ	Nhu cầu kỹ thuật dữ liệu
BR5	Cố vấn học tập	Nhận cảnh báo sớm học viên có nguy cơ bỏ dở	CDC và cửa sổ trượt: đọc log thay đổi từ OLTP, tính điểm rủi ro, phát sinh cảnh báo
BR6	Quản trị viên (B2B)	Theo dõi tiến độ kỹ năng và tỷ lệ hoàn thành theo tổ chức	Pipeline medallion: chuẩn hoá telemetry có phiên bản schema, khử định danh, báo cáo tổ chức với ngưỡng nhóm tối thiểu
BR7	Giảng viên	Xem heatmap tương tác video bài giảng	Tổng hợp chi tiết: mật độ play/pause/tua theo từng giây video
BR8	Giảng viên	Đồng bộ trạng thái, vô hiệu hoá cache, làm mới chỉ mục tìm kiếm khi nội dung thay đổi	Pipeline xu hướng tương tác: tổng hợp hoạt động theo cửa sổ thời gian, tính điểm tương tác, phát hiện xu hướng giảm

Nhận xét về BR8

Cột yêu cầu nghiệp vụ của BR8 (đồng bộ cache và chỉ mục tìm kiếm khi nội dung thay đổi) và cột nhu cầu kỹ thuật (pipeline xu hướng tương tác) mô tả hai nhu cầu khác nhau. Nhóm hiện thực **cả hai**: đồng bộ nội dung dựa trên CDC (Mục 4.1.8.1) và mart xu hướng tương tác (Mục 4.1.8.2).

2.2 Nguồn dữ liệu

Bảng 3 mô tả bốn nguồn dữ liệu thô, đặc tính của chúng và cách nền tảng tiếp nhận. Nguồn nào cũng có một thách thức riêng, và thách thức đó quyết định thiết kế của tầng tiếp nhận.

Bảng 3. Bốn nguồn dữ liệu và cách tiếp nhận

Nguồn	Đặc tính	Thách thức	Tiếp nhận
Telemetry (web, Android, iOS)	JSON bán cấu trúc kiểu xAPI [3], lưu lượng lớn	Đến trễ, sai thứ tự (thiết bị offline), gửi lại, nhiều phiên bản schema, chứa PII, đồng hồ thiết bị lệch	Topic `telemetry.events.raw.v1`, khoá theo người học; DLQ `telemetry.events.dlq.v1`

Nguồn	Đặc tính	Thách thức	Tiếp nhận
CDC từ PostgreSQL	Thay đổi cấp dòng từ WAL: tiến độ, bài nộp, điểm	Giữ thứ tự theo khoá, không ảnh hưởng truy vấn production, xử lý cập nhật và xoá	Debezium (plugin `pgoutput`, replication slot) vào 11 topic `cdc.lms.public.*`
Kho học liệu (object storage)	Transcript, slide, PDF; văn bản dài	Chia đoạn theo ngữ nghĩa, quản lý phiên bản nghiêm ngặt	Đối tượng bất biến theo phiên bản trong MinIO; thay đổi đi qua CDC của bảng `course_content`
Nhật ký sandbox	stdout, stderr, thời gian chạy, bộ nhớ, trạng thái assertion	Chuẩn hoá đầu ra, tách lỗi hạ tầng khỏi lỗi của người học	Topic `grading.submission_received.v1`, `grading.sandbox_result.raw.v1`, `grading.submission_graded.v1`

Danh mục topic Kafka gồm 26 topic, được khai báo dưới dạng mã trong `infra/kafka/topics.yaml`. Ngoài các topic ở trên còn có các topic sự kiện miền: `lms.*` (bài tập mới, sắp đến hạn, khoá học cập nhật), `risk.*` (điểm rủi ro, cảnh báo), `notification.*` và `content.*`.

2.3 Yêu cầu phi chức năng

Từ đặc tính của nguồn dữ liệu và nhu cầu của người dùng, nhóm rút ra các yêu cầu phi chức năng sau. Chúng là tiêu chí để đánh giá các quyết định kiến trúc ở Mục 3.1.

Độ trễ phù hợp từng BR

Thông báo, phản hồi chấm bài và cảnh báo rủi ro cần độ trễ tính bằng giây. Báo cáo tổ chức và heatmap chấp nhận cập nhật theo ngày, kèm lớp phủ cho hôm nay. Tra cứu đặc trưng cho gợi ý cần dưới 10 ms.

Đúng một lần, idempotent

Gửi lại từ Kafka, chạy lại job và backfill không được làm sai số liệu. Mọi lệnh batch phải idempotent theo ngày chạy.

Xử lý theo thời gian sự kiện

Cửa sổ thời gian tính theo lúc sự kiện xảy ra, có watermark. Dữ liệu trễ vẫn được lưu và gắn cờ, không bị bỏ qua âm thầm.

Quyền riêng tư

Định danh thật chỉ tồn tại ở lớp thô và bị hạn chế truy cập. Các lớp phân tích chỉ dùng bút danh. Báo cáo tổ chức không công bố số liệu của nhóm dưới $k = 5$ người.

Tiến hoá schema

Payload của client có nhiều phiên bản; hệ thống phải chuyển đổi về phiên bản hiện hành hoặc từ chối có lý do.

Tái lập và kiểm toán

Có thể dựng lại toàn bộ dữ liệu từ lớp thô, xem lại trạng thái cũ (time travel), và truy vết mỗi lần build.

Chất lượng dữ liệu

Có cổng kiểm tra sau mỗi lớp; lỗi nghiêm trọng phải dừng pipeline.

Chạy được trên máy cá nhân

Toàn bộ hệ thống chạy bằng Docker Compose, chỉ dùng CPU, không phụ thuộc dịch vụ trả phí.

2.4 Ánh xạ yêu cầu vào pipeline

Bảng 4 cho thấy mỗi BR được giao cho bounded context nào, chạy trên runtime nào và được phục vụ từ đâu. Cột cuối nêu mức độ hoàn thiện thực tế; Mục 6.1 giải thích thêm.

Bảng 4. Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime

BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
1	notification	Rule engine trên sự kiện Kafka	Redis (dedup, giới hạn tần suất)	Logic hoàn chỉnh; runtime demo
2	recommendation	Luật tại thời điểm truy vấn; đặc trưng lô `features.learner_longterm`	Delta (ngoại tuyến), API	Rút gọn về rule-based
3	grading	Hàng đợi Kafka, sandbox, `FeedbackPolicy`	API	Logic hoàn chỉnh; runtime demo
4	analytics + br45	CDC, Spark SQL/Delta; PostgreSQL SQL thay thế	Bản công bố Delta và mart PostgreSQL; mart ngày của kho dùng quy tắc riêng	Chạy thật

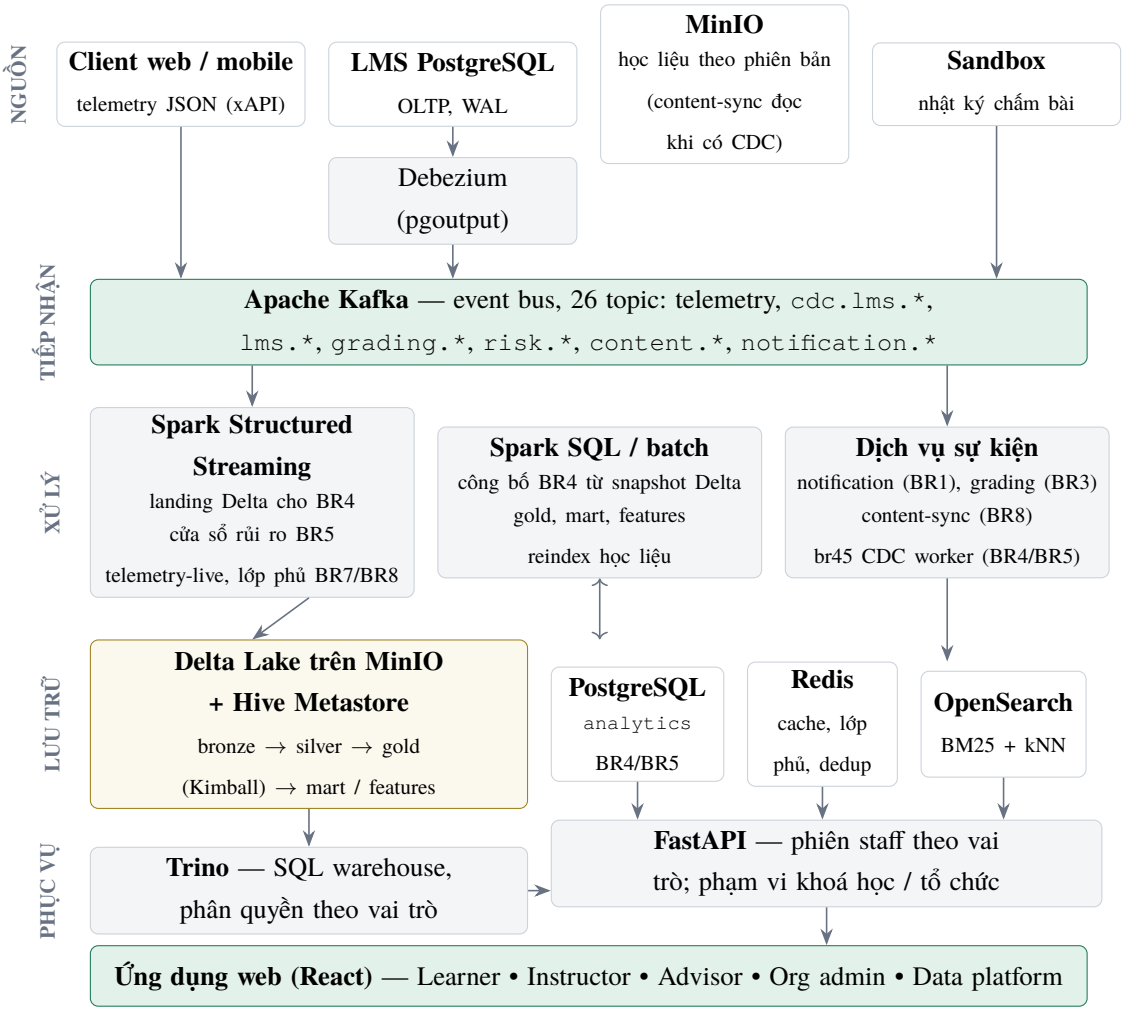
BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
5	risk + br45	CDC, Spark Structured Streaming; Flink đối chiếu	Cửa sổ, điểm và cảnh báo PostgreSQL; outbox cấp lịch sử cho kho	Chạy thật
6	telemetry + analytics	Medallion Spark	Delta, Trino (`mart.org_progress`)	Chạy thật
7	analytics	Spark batch + lớp phủ streaming	Delta, Trino, Redis	Chạy thật
8	content + analytics	CDC, dịch vụ content-sync; Spark batch + lớp phủ	Redis, OpenSearch, Delta, Trino	Chạy thật

3 Kiến trúc, công nghệ và kho dữ liệu

3.1 Kiến trúc giải pháp

3.1.1 Tổng quan kiến trúc

Hình 1 trình bày kiến trúc đầu cuối của nền tảng, theo sáu tầng từ trên xuống. Tầng nguồn gồm bốn nguồn dữ liệu. Tầng tiếp nhận dùng Kafka làm event bus duy nhất, với Debezium bắt thay đổi từ WAL của PostgreSQL. Tầng xử lý dùng Spark cho cả luồng và lô, cùng các dịch vụ tiêu thụ sự kiện. Tầng lưu trữ gồm lakehouse Delta Lake, PostgreSQL cho các mart thời gian thực của BR4/BR5, Redis và OpenSearch. Tầng phục vụ gồm Trino (SQL) và một API FastAPI. Trên cùng là ứng dụng web cho năm nhóm người dùng.



Hình 1. Kiến trúc tổng thể của nền tảng DE_261

Ba nguyên tắc xuyên suốt kiến trúc:

- **Kafka là ranh giới giữa các context.** Các bounded context chỉ giao tiếp qua topic Kafka và bảng lakehouse, không dùng chung cơ sở dữ liệu. Nhờ đó mỗi context có thể triển khai và thay đổi độc lập.
- **Spark là bên ghi lakehouse** (ADR 0011). Spark ghi dữ liệu hạ cánh và snapshot BR4 vào Delta. Worker BR4/BR5 ghi các mart phục vụ và trạng thái xử lý vào PostgreSQL. Lớp phủ tương tác ghi giá trị tuyệt đối vào Redis.
- **Lakehouse là nguồn sự thật cho phân tích.** Redis và các mart phục vụ đều dựng lại được từ lakehouse hoặc từ dữ liệu thô (ADR 0006).

3.1.2 Lựa chọn công nghệ

Bảng 5 liệt kê công nghệ cho từng vai trò. Các thành phần chính gồm Kafka [4], Debezium [5], Spark [6] với Structured Streaming [7], Delta Lake [8], Trino (hậu duệ của Presto [9]), Redis [10]

và OpenSearch [11]. Nhóm giữ các lựa chọn của đề bài khi chúng chạy được trên máy cá nhân. Khi phải thay thế, nhóm chọn thành phần đóng cùng vai trò và ghi lại bằng ADR.

Bảng 5. Công nghệ theo vai trò

Vai trò	Công nghệ	Lý do chọn
Tiếp nhận, event bus	Apache Kafka	Bộ đệm cho telemetry tốc độ cao, giữ thứ tự theo khoá, phát lại được; dùng chung làm event bus cho các sự kiện LMS (ADR 0003)
CDC	Debezium (pgoutput)	Đọc replication slot nên không tạo tải truy vấn lên production; giữ LSN để sắp thứ tự
Xử lý luồng	Spark Structured Streaming	Watermark theo event time, checkpoint, sink giao dịch Delta; dùng chung engine với batch (ADR 0009, 0014)
Xử lý lô	Apache Spark 3.5	Hệ sinh thái Delta, xử lý song song cho model SQL và chia đoạn học liệu (ADR 0004)
Lakehouse	Delta Lake 3.3 trên MinIO	ACID MERGE, time travel, Change Data Feed; tương thích Databricks (ADR 0005)
Catalog	Hive Metastore 3.1.3	Danh mục chung cho Spark và Trino; bản 3.1 tương thích Spark 3.5 (ADR 0010)
SQL warehouse	Trino	Truy vấn trực tiếp bảng Delta, ANSI SQL, phân quyền theo schema, không cần bản sao thứ hai (ADR 0010)
Phục vụ độ trễ thấp	Redis	Tra cứu theo khoá dưới mili-giây, TTL; dùng cho cache, lớp phủ thời gian thực, dedup thông báo (ADR 0006)
Mart phục vụ BR4/BR5	PostgreSQL 16	API đọc bản đã công bố. Trạng thái job và độ tươi cho biết khi nào nguồn đã được xử lý
Tìm kiếm	OpenSearch 2.17	BM25 kết hợp vector HNSW; xếp hạng lai bằng reciprocal rank fusion (ADR 0013)
Điều phối	Apache Airflow	DAG theo lịch và theo dataset; mọi task idempotent theo ngày chạy
API, giao diện	FastAPI, React 18 + TypeScript	Một API chung cho mọi BR; giao diện không dùng thư viện UI hay biểu đồ ngoài

3.1.3 Kiến trúc phần mềm: DDD và kiến trúc lục giác

Toàn bộ mã nằm trong một gói Python `edu_platform` (monorepo, ADR 0002), chia thành bảy bounded context theo Domain-Driven Design [12]: `telemetry`, `notification`, `recommendation`, `grading`, `risk`, `analytics` và `content`. Mỗi context có bốn lớp theo kiến trúc lục giác [13] như Bảng 6. Một test kiến trúc (`tests/unit/test_architecture.py`) kiểm tra quy tắc phụ thuộc giữa các lớp.

Bảng 6. Các lớp trong một bounded context

Lớp	Chứa gì	Được phụ thuộc vào
`domain/`	Thực thể, value object, aggregate, sự kiện miền, *policy* (ví dụ `EngagementPolicy`, `MinCohortPolicy`)	Chỉ `shared_kernel.domain`
`application/`	Use case (mỗi lớp một hàm `execute`), các cổng `Protocol`	domain
`infrastructure/`	Adapter: Kafka, Redis, Delta, S3, Trino, OpenSearch, YAML	Mọi thứ
`entrypoints/`	Điểm ghép nối cho job Spark, dịch vụ, API	Mọi thứ

Lợi ích lớn nhất của cách tổ chức này là **một logic, nhiều engine**. Ví dụ, use case `NormalizeEvents` (kiểm tra hợp lệ, chuyển schema, khử định danh, gắn cờ trễ) chạy trong job streaming thời gian thực, trong job silver của lakehouse, và trong runtime demo. Kết quả luôn giống nhau, kể cả bút danh và quyết định đưa vào DLQ. Các ngưỡng nghiệp vụ dùng trong SQL của mart cũng được lấy từ value object của domain, và có test đơn vị giữ hai phía đồng bộ.

3.1.4 Ngữ nghĩa xử lý luồng

- **Thời gian sự kiện ở mọi nơi** theo mô hình Dataflow [14]. Watermark cho phép sự kiện đến muộn tối đa 5 phút với telemetry và 10 giây với CDC. Riêng cửa sổ rủi ro BR5 dùng watermark 21 ngày, vì dữ liệu học tập có thể được đồng bộ muộn.
- **Dữ liệu trễ không bị mất.** Sự kiện trễ hơn watermark vẫn được lưu và gắn cờ. Các model gold tự mở rộng cửa sổ tính toán về ngày của sự kiện trễ nhất (cơ chế `late_data`, Mục 3.2.8).
- **Đúng một lần.** Landing dùng checkpoint của Structured Streaming kết hợp sink giao dịch Delta. Lớp silver dùng MERGE idempotent theo khoá nghiệp vụ. Lớp phủ Redis ghi giá trị tuyệt đối thay vì cộng dồn, nên phát lại không đếm hai lần.

- **Thứ tự CDC.** Thay đổi được sắp theo `source.lsn` cho từng khoá. Thay đổi có LSN cũ hơn không thể ghi đè bản ghi mới hơn.
- **Đồng hồ thiết bị lệch.** Sự kiện có thời gian sau thời điểm tiếp nhận quá 2 phút được ghim về thời điểm tiếp nhận (`ClockSkewPolicy`). Nếu không, một thiết bị có đồng hồ chạy nhanh sẽ đẩy watermark lên và làm rơi sự kiện đúng hạn của mọi người khác.

3.1.5 Các quyết định kiến trúc

Nhóm ghi mỗi quyết định quan trọng thành một ADR [15] trong `docs/adr/`. ADR đã chấp nhận thì không sửa; quyết định mới thay thế quyết định cũ bằng một ADR mới. Bảng 7 tóm tắt mười bốn ADR.

Bảng 7. Tóm tắt các quyết định kiến trúc (ADR)

ADR	Quyết định	Lý do và hệ quả chính
0001	Ghi quyết định bằng ADR	Tránh tranh luận lại; lưu lý do, không chỉ kết quả
0002	Monorepo Python, DDD + hexagonal	Use case độc lập engine; một chi phí nhỏ cho các lớp gián tiếp
0003	Kafka + Debezium cho tiếp nhận và CDC	Giữ thứ tự theo thực thể; phải giám sát replication slot
0004	Flink cho luồng, Spark cho lô	Lựa chọn ban đầu theo đề bài; được điều chỉnh bởi ADR 0009 và 0014
0005	Lakehouse Delta Lake medallion	ACID upsert, time travel, ranh giới PII; cần job bảo trì chống file nhỏ
0006	Redis cho tầng phục vụ	Dưới mili-giây, TTL; Redis không phải nguồn sự thật
0007	Khử định danh ở ranh giới silver	HMAC-SHA256 có phiên bản khoá; báo cáo tổ chức áp dụng $k = 5$
0008	Runtime demo trong tiến trình	Trình diễn đầu cuối trên laptop bằng adapter trong bộ nhớ, cùng các cổng như production
0009	Mart PostgreSQL cho BR4/BR5; Spark SQL/Delta và Spark Structured Streaming	Phục vụ theo giao dịch; BR4 có snapshot bất biến; BR5 có đối chiếu Flink
0010	Trino + Hive Metastore làm SQL warehouse	Đóng vai “Databricks SQL + catalog” trên chính các bảng Delta, không có bản sao thứ hai

ADR	Quyết định	Lý do và hệ quả chính
0011	Spark là bên ghi duy nhất của Delta	Connector Flink cho Delta đã lỗi thời; tránh xung đột commit
0012	Gold là lược đồ hình sao Kimball, khoá tất định	`xxhash64` của khoá tự nhiên; dựng lại tái tạo đúng khoá; thành viên suy luận cho chiều đến muộn
0013	Đồng bộ nội dung BR8 bằng CDC vào OpenSearch	Một đường đồng bộ cho mọi bên ghi; idempotent theo phiên bản đã chỉ mục
0014	Spark Structured Streaming cho telemetry nhanh và lớp phủ BR7/BR8	Các job Flink chưa tồn tại; độ trễ đo được khoảng 7 giây đủ cho lớp phủ

3.1.6 Điều chỉnh so với kiến trúc tham chiếu của đề bài

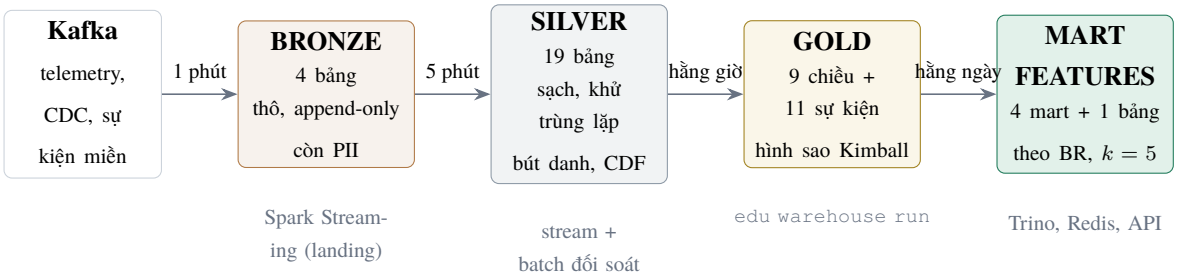
Nhóm giữ nguyên các vai trò trong kiến trúc của đề bài, nhưng có bốn điểm điều chỉnh có chủ đích:

1. **Spark cho đường triển khai BR4/BR5** (ADR 0009, 0014). BR4 dùng Spark SQL trên snapshot Delta và ghi mart phục vụ vào PostgreSQL. PostgreSQL SQL xử lý cùng số đếm cơ sở từ dữ liệu đã chuẩn hóa và có thể phục vụ thay thế. BR5 dùng Spark Structured Streaming để duy trì cửa sổ. Job Flink [16] đã hiện thực cho phép đối chiếu cửa sổ và checkpoint trên cùng đầu vào. Kết quả và giới hạn của từng phép đo nằm ở Mục 5.
2. **Trino + Hive Metastore cho vai trò "Databricks"** (ADR 0010). Delta Lake chỉ là định dạng bảng. Vai trò danh mục và SQL warehouse của Databricks được lấp bằng Hive Metastore và Trino, chạy được cục bộ. Trên cloud có thể thay bằng Databricks SQL và Unity Catalog mà không đổi mô hình dữ liệu.
3. **Redis thay Cassandra** cho tầng phục vụ (ADR 0006). Đề bài cho phép chọn một trong hai. Redis đủ cho khối lượng hiện tại; adapter nằm sau cổng nên có thể thay khi dữ liệu vượt bộ nhớ.
4. **BR2 rút gọn về gợi ý dựa trên luật.** Đề bài mô tả pipeline đặc trưng hai nhánh qua feature store. Nhóm đã có nhánh lô (`features.learner_longterm`). Nhánh đặc trưng phiên và mô hình học máy chưa được hiện thực; gợi ý hiện được tính bằng luật tại thời điểm truy vấn (Mục 4.1.2).

3.2 Lakehouse và kho dữ liệu

Mục này trình bày phần cốt lõi phục vụ phân tích: lakehouse [17] Delta Lake tổ chức theo mô hình medallion [18], và kho dữ liệu chính là lớp gold của lakehouse. Nền tảng không có cơ sở dữ liệu phân tích thứ hai. Spark ghi các bảng Delta, Hive Metastore lập danh mục, Trino phục vụ truy vấn SQL, còn Redis phục vụ các tra cứu cấp mili-giây. Toàn bộ DDL, model biến đổi, cổng chất lượng và cấu hình nằm dưới dạng mã trong thư mục `warehouse/`.

3.2.1 Kiến trúc medallion



Hình 2. Các lớp của lakehouse và tần suất cập nhật

Bảng 8. Hợp đồng dữ liệu của từng lớp

Schema	Số bảng	Bên ghi	Hợp đồng
`bronze`	4	Stream landing	Mỗi bản ghi Kafka một dòng, giữ nguyên như nhận được. Chỉ append, còn PII, chỉ vai trò `pii_steward` đọc được. Giữ 30 ngày (telemetry) hoặc 90 ngày (CDC, sự kiện). Phát lại bronze dựng lại được mọi lớp sau.
`silver`	19	Stream silver + batch đối soát	Có kiểu, đã kiểm tra hợp lệ, khử trùng lặp, đúng schema hiện hành, đã khử định danh bằng HMAC; bỏ định danh trực tiếp. CDC áp dụng theo thứ tự LSN, xoá mềm. Bật Change Data Feed.
`gold`	9 + 11	`edu warehouse run`	Lược đồ hình sao Kimball (ADR 0012): bảng chiều dùng chung, khoá thay thế tất định, bảng sự kiện có grain rõ ràng.
`mart`	4	`--layer mart`	Mỗi BR một bảng, theo nhu cầu người đọc; ngưỡng lấy từ policy của domain. `mart.org_progress` là bảng duy nhất quản trị viên B2B được đọc.

Schema	Số bảng	Bên ghi	Hợp đồng
`features`	1	`--layer features`	Kho đặc trưng ngoại tuyến cho BR2, mỗi `as_of_date` một phân vùng để huấn luyện đúng thời điểm (point-in-time).
`ops`	2	Runner	`pipeline_runs` (mỗi lần build: cửa sổ, số dòng, phiên bản Delta) và `dq_results` (kết quả kiểm tra chất lượng).

3.2.2 Chuẩn hoá telemetry và tiến hoá schema

Telemetry đi qua use case `NormalizeEvents`, chạy giống hệt nhau trong job silver và trong job thời gian thực:

1. **Kiểm tra hợp lệ** theo hợp đồng trong `schemas/`: thiếu trường bắt buộc, động từ (verb) không thuộc từ vựng, hoặc payload hỏng thì bị đưa vào DLQ kèm lý do.
2. **Chuyển schema**: mỗi sự kiện mang `schema_version`. Chuỗi `SchemaMigrator` đưa phiên bản cũ (1.0) lên phiên bản hiện hành (1.1); phiên bản không biết bị từ chối.
3. **Khử định danh**: định danh người học được thay bằng bút danh; email, họ tên, IP, mã thiết bị bị loại bỏ.
4. **Thời gian**: tính độ trễ so với lúc tiếp nhận, gắn cờ `late`, và ghim các sự kiện có đồng hồ chạy trước.
5. **Khử trùng lặp**: MERGE theo `event_id` (chỉ chèn) loại các lần gửi lại từ thiết bị offline.

3.2.3 Khử định danh và phân quyền

Định danh thật chỉ tồn tại ở bronze (ADR 0007). Từ silver trở đi, người học được biểu diễn bằng bút danh dạng `k<phiên bản khoá>_<24 ký tự hex>`, tính bằng HMAC-SHA256 với khoá bí mật có phiên bản. Hàm SQL `pseudonymize()` dùng đúng thuật toán và khoá của `HmacPseudonymizer` trong context telemetry, nên dữ liệu telemetry, CDC và sự kiện miền vẫn nối được với nhau. Cổng chất lượng `silver_no_raw_identifiers` chặn mọi định danh thô lọt qua bronze.

Trino áp dụng phân quyền chỉ đọc theo schema (Bảng 9). Quản trị viên B2B chỉ thấy đúng một bảng đã áp dụng k-anonymity.

Bảng 9. Phân quyền truy cập kho dữ liệu qua Trino

Người dùng	bronze	silver	gold	mart	features / ops
`pii_steward`	✓	✓	✓	✓	✓
`analyst`	–	✓	✓	✓	✓
`app` (API, dashboard)	–	–	✓	✓	–
`b2b_admin`	–	–	–	chỉ `org_progress`	–

3.2.4 Mô hình chiều Kimball ở lớp gold

Tám yêu cầu đọc các sự kiện chồng lấn nhau (sự kiện học tập, bài nộp, ghi danh, rủi ro) theo cùng các thực thể (người học, khoá học, nội dung, bài tập, tổ chức). Nếu không có bảng chiều dùng chung, mỗi mart sẽ tự suy ra các thực thể này và dần lệch nhau. Vì vậy lớp gold được mô hình hoá thành lược đồ hình sao theo phương pháp Kimball [19] (ADR 0012). Bảng 10 là ma trận bus cho biết mỗi bảng sự kiện dùng những chiều nào.

Bảng 10. Ma trận bus của lớp gold (D: vai trò ngày, ●: khoá chiều)

Bảng sự kiện (grain)	*date*	*learner*	*course*	*content*	*activity*	*lesson*	*assignment*	*test case*	*org*	BR
`fct_learning_event` (một sự kiện)	D	●	●	●	●					6,7,8
`fct_learner_daily_activity` (người học × khoá × ngày)	D	●	●							2,8
`fct_video_second_activity` (phiên bản × giây × ngày)	D			●						7
`fct_submission` (một lần nộp)	D	●	●				●			3,4
`fct_test_case_result` (lần nộp × test)	D	●					●	●		4
`fct_lesson_progress` (người học × bài)		●	●			●				6
`fct_enrollment` (một lần ghi danh)	D×5	●	●						●	5,6
`fct_risk_score_daily` (người học × khoá × ngày)	D	●	●							5
`fct_dropout_alert` (một cảnh báo)	D	●	●							5
`fct_notification` (một kết quả gửi)	D	●								1
`fct_content_sync` (một phiên bản nội dung)			●	●						8

3.2.4.1 Bảng chiều và khoá thay thế

- **SCD loại 2** cho `dim_learner` và `dim_content`. Với `dim_learner`, báo cáo B2B phải quy hoạt động về tổ chức tại thời điểm xảy ra. Với `dim_content`, mỗi phiên bản video có một dòng riêng; video quay lại có độ dài khác nên giây thứ n của hai phiên bản không cùng nghĩa (BR7). Các chiều còn lại dùng SCD loại 1.
- **Khoá thay thế tất định**: `xxhash64(khoá tự nhiên)` với SCD1 và `xxhash64(khoá tự nhiên, valid_from)` với SCD2. Dựng lại một bảng chiều sẽ tạo lại đúng các khoá cũ, nên bảng chiều và bảng sự kiện build độc lập, không cần dịch vụ sinh số tuần tự.

- **Thành viên suy luận** (inferred member) cho chiều đến muộn: telemetry của một người học mà dòng CDC chưa đến sẽ tạo một dòng `is_inferred = true`, mang đúng khoá mà dòng thật sẽ có. Khi dòng thật đến, nó ghi đè tại chỗ nên không bảng sự kiện nào phải đổi khoá. Đã kiểm chứng: người học `l-131` có sự kiện hai ngày trước dòng CDC vẫn giữ nguyên khoá qua quá trình chuyển đổi.
- Khoá `-1` biểu diễn "không áp dụng", ví dụ nội dung của một sự kiện đăng nhập.

Một số quy tắc nghiệp vụ dùng chung: bài học được coi là hoàn thành khi CDC báo `percent_done = 100` *hoặc* có sự kiện telemetry `completed`, tuỳ cái nào đến trước; vị trí video vượt quá độ dài phiên bản được kẹp về giây cuối; lần chạy sandbox mới nhất của một bài nộp thay thế các lần trước.

3.2.5 Data mart theo yêu cầu nghiệp vụ

Mỗi mart hiện thực lại một use case phân tích bằng SQL theo tập hợp. Các ngưỡng được truyền từ value object của domain (`policy_params`), ghi lại trên mỗi dòng và có test đơn vị đối chiếu với domain.

Bảng 11. Các data mart và kho đặc trưng

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`mart.assessment_item_stats`	BR4, mart ngày	Tỷ lệ đạt theo (bài tập, test case) trong 28 ngày; bất thường khi có ≥ 30 lượt, tỷ lệ đạt $\leq 0{,}35$ và thấp hơn trung vị của bài tập ít nhất $0{,}30$	`AnomalyThreshold`
`mart.org_progress`	BR6, quản trị B2B	Theo tổ chức × khoá học: quy mô nhóm, tỷ lệ hoàn thành, mức thành thạo trung bình; **nhóm dưới k người chỉ công bố quy mô** [20]	`MinCohortPolicy(k=5)`
`mart.video_heatmap`	BR7, giảng viên	Lưới dày theo từng giây cho mỗi phiên bản video trong 28 ngày; số người học phân biệt chính xác; cường độ 0–1	–
`mart.engagement_trend`	BR8, giảng viên	4 cửa sổ tuần cho mỗi người học × khoá; "giảm" khi điểm giảm $\geq 30\%$ trong mỗi cửa sổ của hai tuần gần nhất	`EngagementWeights`, tỷ lệ 0,30

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`features.learner_longterm`	BR2, gợi ý	`ComputeBatchFeatures` trong 30 ngày, theo `as_of_date`	registry đặc trưng v1

Mart ngày BR4 trong bảng này có cửa sổ và ngưỡng riêng. API Assessment Health đọc ba mart PostgreSQL của đường BR4 trình bày ở Mục 4.1.4. Với BR5, runtime giữ cửa sổ, điểm và cảnh báo đang phục vụ ở PostgreSQL. Outbox phát `risk.learner_score.v1` và dữ liệu chuyển trạng thái cho các model lịch sử của kho. ID của lần đánh giá phân biệt các lần cùng điểm nhưng khác policy hoặc lý do. Kho giữ các bảng sự kiện điểm, cảnh báo và chuyển trạng thái để phân tích sau.

3.2.6 Thiết kế vật lý

- **Tính năng Delta**: ACID MERGE, `INSERT ... REPLACE WHERE` cho cửa sổ ngày, ghi đè phân vùng động, Change Data Feed ở silver, time travel (`FOR VERSION AS OF`) trong phạm vi VACUUM 7 ngày.
- **Phân vùng** chỉ theo cột ngày mà truy vấn lọc (ngày sự kiện, ngày hoạt động, ngày nộp, `as_of_date`); bảng nhỏ không phân vùng; bronze phân vùng theo `ingest_date`.
- **Gom cụm**: thuộc tính bảng `edu.zorder` được job bảo trì hằng ngày áp dụng bằng `OPTIMIZE ... ZORDER BY`. Cùng job này gộp các file nhỏ do writer streaming sinh ra.
- **Chính sách là thuộc tính bảng**: `edu.pii`, `edu.owner`, `edu.retention_days`, `edu.zorder` hiển thị cho mọi engine (`SHOW TBLPROPERTIES`, bảng `$properties` của Trino).
- **Tiến hoá schema tường minh**: tắt `autoMerge`; đổi DDL kèm câu lệnh migration.

3.2.7 Cổng kiểm tra chất lượng dữ liệu

Sau mỗi lớp, runner chạy các truy vấn kiểm tra trong `warehouse/quality` và ghi kết quả vào `ops.dq_results`. Cổng mức `error` dừng lần chạy.

Bảng 12. Các cổng kiểm tra chất lượng

Cổng	Mức	Bảo vệ điều gì
`silver_event_id_unique`	error	Khử trùng lặp các lần gửi lại từ mobile
`silver_no_raw_identifiers`	error	Ranh giới PII: chỉ bút danh sau bronze
`bronze_dlq_rate`	warn	Hơn 5% telemetry bị từ chối
`gold_dim_learner_scd2`	error	Mỗi người học đúng một phiên bản hiện hành, không chồng lấn hiệu lực
`gold_orphan_keys`	error	Mọi khoá trong bảng sự kiện tồn tại trong bảng chiều
`gold_unresolved_learners`	warn	Sự kiện bị quy về thành viên "không rõ"
`mart_org_progress_k_anonymity`	error	Không công bố tỷ lệ cho nhóm dưới k người
`mart_rates_in_range`	error	Các tỷ lệ nằm trong $[0, 1]$
`mart_org_progress_unresolved_dims`	warn	Dòng báo cáo thiếu khoá học hoặc kỹ năng
`mart_video_heatmap_unversioned`	warn	Ô heatmap không gắn phiên bản video

3.2.8 Điều phối, dữ liệu trễ và chạy lại

Các job streaming và dịch vụ chạy liên tục. Các job lô được Airflow điều phối (Bảng 13). Mọi lệnh lô đều idempotent theo `--run-date`, nên Airflow retry hay backfill đều an toàn, và `--start-date` dựng lại được bất kỳ khoảng thời gian nào.

Bảng 13. Các DAG Airflow

DAG	Lịch	Công việc
`medallion_daily`	hằng giờ	đối soát silver, build gold
`analytics_marts_daily`	02:00 hằng ngày	làm mới gold, mart BR4 / BR6 / BR7 cho ngày đã đóng
`engagement_weekly`	thứ Hai 02:00	`mart.engagement_trend`
`features_daily`	khi gold cập nhật	`features.learner_longterm`
`content_embedding`	6 giờ một lần	chia đoạn, nhúng học liệu
`lakehouse_maintenance`	04:00 hằng ngày	xoá theo thời hạn lưu, OPTIMIZE / Z-order, VACUUM

Bảng 14. Cơ chế xử lý dữ liệu trễ, trùng lặp và sai thứ tự

Tình huống	Lớp	Cơ chế
Kafka gửi lại, job chạy lại	bronze	Checkpoint Structured Streaming + sink Delta (đúng một lần)
Client gửi lại (mobile offline)	silver	MERGE theo `event_id`; cổng `silver_event_id_unique`
CDC sai thứ tự hoặc bị phát lại	silver	Lấy thay đổi mới nhất theo `lsn`; chỉ MERGE khi `lsn` mới hơn; phát lại không tạo commit Delta
Xoá ở OLTP	silver	Debezium chế độ `rewrite`: `is_deleted = true` (xoá mềm)
Sự kiện trễ (mobile đồng bộ sau vài ngày)	gold	Model theo cửa sổ khai báo `late_data`; cửa sổ tự lùi về ngày sự kiện cũ nhất mà silver nhận kể từ lần build trước (đã kiểm chứng: lần chạy 06–14/10 tự mở rộng về 04/10)
Sự kiện đến trước chiều	gold	Thành viên suy luận với khoá cuối cùng (ADR 0012)
Xoá theo thời hạn lưu ở bronze	silver	`skipChangeCommits` để thao tác xoá không làm dừng stream

Tình huống	Lớp	Cơ chế
Tiến hoá schema của client	silver	Chuỗi `SchemaMigrator` 1.0 → 1.1; không biết thì vào DLQ

Khi có sự cố, quy trình khôi phục dựa trên bronze là nhật ký phát lại: sửa SQL rồi chạy lại `edu warehouse run --start-date` cho lớp bị ảnh hưởng; khi cần trạng thái cũ thì dùng time travel của Delta với phiên bản lưu trong `ops.pipeline_runs`.

4 Hiện thực và giao diện ứng dụng

4.1 Thiết kế và hiện thực theo từng yêu cầu

BR4–BR8 chạy trên PostgreSQL, Debezium, Kafka, Spark, Delta, Trino, Redis và OpenSearch. Giao diện BR1–BR3 dùng runtime demo (ADR 0008). Code Lab Classroom còn lưu kết quả và hoạt động vào PostgreSQL để chuyển qua CDC. Bảng 21 nêu mức kiểm chứng của từng luồng.

4.1.1 BR1 — Pipeline thông báo hướng sự kiện

Luồng. Hệ thống LMS phát các sự kiện `lms.assignment_created.v1`, `lms.deadline_approaching.v1`, `lms.course_updated.v1`; các context khác phát `grading.submission_graded.v1` và `risk.dropout_risk_detected.v1`. Dịch vụ thông báo tiêu thụ các topic này, đối chiếu với tập luật, xác định người nhận, rồi gửi qua các kênh in-app, push hoặc email. Kết quả (đã gửi hoặc bị chặn) được phát lên `notification.dispatched.v1` và `notification.suppressed.v1`, rồi đi vào lakehouse thành `gold.fct_notification`.

Rule engine. Luật được khai báo trong YAML và nạp lại nóng được (Hình 3). Điều kiện viết bằng một ngôn ngữ biểu thức rất nhỏ trên payload của sự kiện, không cho chạy mã tuỳ ý.

```
rules:
  - rule_id: deadline-24h
    event_type: lms.deadline_approaching.v1
    condition: "hours_to_deadline <= 24"
    channels: [push, email]
    priority: high
    template_id: deadline_soon
  - rule_id: dropout-risk-advisor
    event_type: risk.dropout_risk_detected.v1
    channels: [email]      # ườngi ậnhn = ốc ấvn ủca ọhc viên
    priority: high
```

Hình 3. Trích tập luật thông báo (`infra/notification/rules.yaml`)

Chống làm phiền. Mỗi thông báo có một khoá khử trùng lặp, lưu trong Redis với TTL 24 giờ, nên cùng một sự kiện không bao giờ gửi hai lần. Chính sách giới hạn tần suất mặc định là tối đa 5 thông báo mỗi giờ cho mỗi người học. Thông báo ưu tiên cao (ví dụ còn dưới 24 giờ đến hạn) được bỏ qua giới hạn này. Thông báo bị chặn được ghi lại kèm lý do để giao diện hiển thị.

Mức kiểm chứng. Bộ luật, chính sách và use case có kiểm thử. Dịch vụ Kafka và Redis có cấu hình chạy riêng. Hộp thông báo trên web hiện đọc dữ liệu demo, còn đường từ dịch vụ Kafka đến hộp thông báo cần nghiệm thu.

4.1.2 BR2 — Gợi ý khoá học và bước học tiếp theo

Đề bài mô tả một pipeline đặc trưng hai nhánh: đặc trưng dài hạn tính theo lô bằng Spark, kết hợp đặc trưng phiên tính theo luồng, qua feature store. Nhóm đã hiện thực nhánh lô: bảng `features.learner_longterm` gồm tỷ lệ hoàn thành 30 ngày, tỷ lệ đạt trung bình, số ngày hoạt động, nền tảng ưa dùng và mức thành thạo theo kỹ năng. Bảng có một phân vùng cho mỗi ngày tính, để huấn luyện mô hình sau này không bị rò rỉ dữ liệu tương lai.

Bộ gợi ý hiện tại được rút gọn thành **gợi ý dựa trên luật**, tính tại thời điểm truy vấn từ danh mục khoá học và các khoá học viên đã hoàn thành. Luật xếp các khoá chưa ghi danh theo *tầng*, sau đó theo mức liên quan, độ khó và độ ưa thích (Bảng 15). Mỗi gợi ý luôn kèm một câu giải thích để người học hiểu vì sao.

Bảng 15. Các tầng của luật gợi ý (BR2)

Tầng	Trường hợp	Ví dụ lời giải thích	Điểm
0	Bước tiếp theo trong lộ trình	"Tiếp tục lộ trình data engineering: mức khó tiếp theo" hoặc "Bước liên quan trong lộ trình"	1,00
1	Mở rộng cùng mức	"Mở rộng ở mức của bạn: một khoá analytics mức dễ"	0,75
2	Bù khoá bị bỏ qua	"Bổ sung khoá mức dễ bạn đã bỏ qua"	0,50
3	Quá sức	"Quá sức: hoàn thành khoá mức trung bình trước" (khoá bị khoá)	0,00

4.1.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết

Luồng Kafka riêng. Dịch vụ tiếp nhận đưa bài nộp vào `grading.submission_received.v1`. Worker sandbox chạy bài và ghi nhật ký lên `grading.sandbox_result.raw.v1`. Dịch vụ chấm bài chuẩn hoá kết quả và phát `grading.submission_graded.v1` cho luồng thông báo BR1.

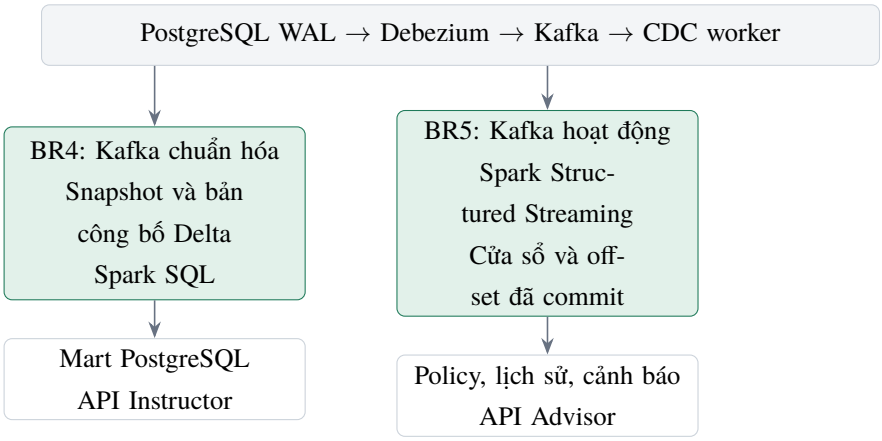
Code Lab trong Classroom. Giao diện dùng bộ chấm demo và lưu bài nộp cùng các kết quả test trong PostgreSQL. Một bài FizzBuzz kiểm chứng cùng ID xuất hiện trong phản hồi, bảng nguồn và facts BR4 với đủ năm test. Hai hoạt động bài học Python trong kịch bản kiểm tra làm điểm rủi ro giảm từ 15 xuống 0. Đường Kafka riêng cần kiểm chứng tích hợp với giao diện.

Sandbox. Bài Python chạy trong tiến trình con có giới hạn thời gian 3 giây. Bài SQL chạy trên SQLite trong bộ nhớ ở chế độ chỉ đọc, có hàm ngắt khi quá hạn. Lỗi hạ tầng được tách khỏi lỗi trong mã của người học: lỗi và timeout của mã người học tính là trượt, còn lỗi hạ tầng không tính vào tỷ lệ đạt.

Phản hồi có cấu trúc. `FeedbackPolicy` là một hàm thuần, không có I/O, tạo phản hồi gồm: câu tóm tắt, danh sách test trượt kèm giá trị mong đợi và thực tế (hoặc diff), 8 dòng cuối của stderr, và gợi ý theo ngôn ngữ dựa trên một bảng mẫu lỗi thường gặp. Kết quả còn gồm thời gian chạy, bộ nhớ và mã thoát. Mục 4.2 cho thấy cách phản hồi này hiển thị trong Code lab.

4.1.4 BR4 — Phát hiện câu hỏi và test case bất thường

Luồng CDC thật (ADR 0009). Dữ liệu vận hành nằm trong schema `br45` của PostgreSQL: khoá học, bài tập, câu hỏi và test case có phiên bản, bài nộp và kết quả chấm. Một connector Debezium riêng bắt WAL vào các topic `br45.cdc.br45.<bảng>`. Worker CDC lưu phong bì gốc vào `analytics.raw_changes`, kiểm tra hợp đồng (sai thì vào `quarantine`), khử trùng lặp theo định danh ổn định gồm bảng, LSN, giao dịch, thao tác và ảnh dòng, rồi ghi `clean_rows`. Thay đổi có LSN cũ không thể làm sống lại bản ghi mới hơn.



Hình 4. Từ CDC đến kết quả phục vụ BR4 và BR5

Phương án cơ sở. Worker PostgreSQL SQL đọc các bản ghi đã chuẩn hóa trong `analytics.clean_rows`, tính chỉ số trong một giao dịch và ghi mart phục vụ. Engine này dùng cùng API job và cùng logic phân loại cơ sở với đường Spark. Phép đối chiếu ở Mục 5 kiểm tra số đếm cơ sở của cả hai engine bằng oracle độc lập.

Đường triển khai mặc định. Worker Spark đọc snapshot Delta theo `versionAsOf` khi chuỗi đã hạ cánh bao phủ revision mà job yêu cầu. Job theo khóa học và cửa sổ có lease. Mỗi lần công bố dùng một định danh từ job, Delta version, source revision và lease generation. Spark ghi facts và các chiều khoá học, câu hỏi, test case, ghi danh vào đường dẫn bất biến của lần công bố. Sau khi kiểm tra chất lượng và xác nhận còn lease, worker thay ba mart PostgreSQL và cập nhật con trỏ công bố trong cùng giao dịch. Nếu revision nguồn đổi trong lúc tính, job quay lại hàng đợi cho revision mới.

Ngữ nghĩa chỉ số. Mọi khoảng quan sát là nửa mở $[t_0, t_1)$. Một lần chấm test hợp lệ khi bài nộp đã được chấm, kết quả hợp lệ và trạng thái thuộc {passed, failed, error, timeout}. Một lần làm câu hỏi hợp lệ khi mọi test bắt buộc của phiên bản câu hỏi *tại thời điểm nộp* đều có kết quả hợp lệ, và chỉ đạt khi mọi test bắt buộc đều đạt. Phiên bản test mới hơn không âm thầm diễn giải lại các bài nộp cũ.

$$\text{pass_rate} = \frac{\text{số lần chấm hợp lệ đạt}}{\text{tổng số lần chấm hợp lệ}}$$

Thiếu kết quả test bắt buộc, đánh giá không hợp lệ và lỗi hạ tầng bị loại khỏi mẫu số và được đếm riêng. Mẫu số bằng 0 cho kết quả `null` (hiển thị là chưa có số liệu). Hệ thống cũng tính tỷ lệ đạt ở lần làm đầu tiên và lần làm cuối cùng của mỗi học viên.

Bảng 16. Quy tắc phát hiện bất thường của BR4 (cấu hình được)

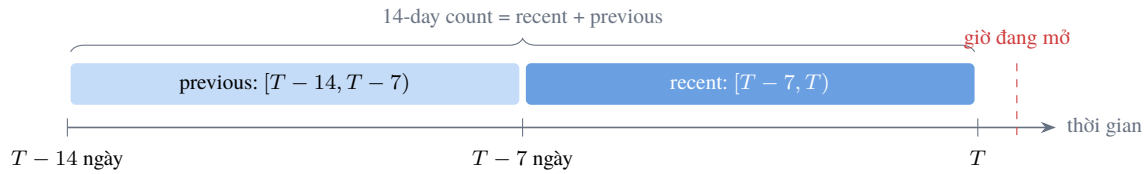
Quy tắc	Mặc định	Trạng thái sinh ra
Số học viên hợp lệ phân biệt tối thiểu	20	`INSUFFICIENT_DATA` nếu thiếu
Tỷ lệ đạt thấp của câu hỏi	dưới 30%	`LOW_PASS_RATE`
Tỷ lệ đạt thấp của test case	dưới 20%	`LOW_PASS_RATE`
Thấp hơn trung vị các test bắt buộc khác	$\geq$ 50 điểm %	`SUSPICIOUS_TEST_CASE`
Giảm so với cửa sổ liền trước cùng độ dài	$\geq$ 30 điểm %	`NEEDS_REVIEW`

Kết quả lưu ở ba mart câu hỏi, test case và bất thường. Đường Spark bổ sung phân tích nhóm học viên có thành tích trước đó từ phân vị 80 trên các câu hỏi khác, giữ các trường hợp đồng hạng và yêu cầu tối thiểu ba câu hỏi có lịch sử. Chỉ hiển thị tỷ lệ nhóm khi đủ số học viên. Cờ bất thường hướng dẫn giảng viên xem lại đề và kết quả chấm.

Ví dụ sửa điểm. Với 24 lượt hợp lệ, hai lượt đạt và 22 lượt trượt cùng một test, giảng viên khoanh vùng test có tỷ lệ 8,33%. Debezium chuyển `UPDATE` của lần chấm qua Kafka. Bước chuẩn hóa áp dụng thay đổi theo LSN và yêu cầu tính lại cửa sổ liên quan. Mart mới trả $3/24$. Lượt thử chèn lỗi sau khi thay mart nhưng trước commit giữ kết quả $2/24$ và Delta version cũ trong API. Lượt retry công bố $3/24$ ở phiên bản Delta mới. Snapshot cũ vẫn đọc được.

4.1.5 BR5 — Cảnh báo sớm nguy cơ bỏ học

Cửa sổ trượt theo thời gian sự kiện. Spark Structured Streaming đọc `br45.activity.v1`, dùng watermark 21 ngày và `dropDuplicatesWithinWatermark(event_id)`, rồi tổng hợp bằng `window(event_time, '7 days', slide)` với bước trượt mặc định 1 giờ. Phương án đối chiếu Flink dùng cùng cửa sổ, watermark và ngữ nghĩa rút lại. Một phép tính lô Python từ các sự kiện gốc kiểm tra số đếm của từng engine. Với mốc đánh giá T căn theo giờ UTC, các đặc trưng được tính như Hình 5.



Hình 5. Các cửa sổ đặc trưng tại mốc đánh giá T (BR5)

Cập nhật một hoạt động sẽ rút phần đóng góp cũ khỏi các cửa sổ rồi cộng phần mới; xoá thì rút phần đóng góp. Sự kiện "đồng hồ" có trọng số 0 tạo các cửa sổ rỗng, để tình trạng không hoạt

động vẫn được ghi nhận khi học viên không làm gì. Mỗi micro-batch upsert tổng tuyệt đối của cửa sổ trong một giao dịch. Việc chấm điểm rủi ro chờ đến khi offset Spark đã commit bao phủ hết outbox, để tránh chấm trên đặc trưng chưa đầy đủ. Khi khởi động lại, checkpoint giữ cả tổng cửa sổ và định danh sự kiện. Phép thử gửi lại một ID cũ và thêm một ID mới kiểm tra đồng thời hai tính chất này. Dữ liệu cũ hơn watermark đi vào vùng cách ly và được xử lý bằng backfill cô lập khi cần.

Điều kiện xét. Học viên phải ghi danh đang hiệu lực ít nhất 7 ngày. Bài tập bắt buộc chỉ tính khi hạn nộp nằm trong 14 ngày gần nhất và sau ngày ghi danh; hạn trong tương lai không bị phạt. Riêng việc thiếu telemetry không bị coi là không hoạt động: tín hiệu này đòi hỏi đã từng quan sát thấy hoạt động thật.

Bảng 17. Các tín hiệu rủi ro và điểm (BR5)

Tín hiệu	Điều kiện	Điểm
Không hoạt động	Ít nhất 7 ngày kể từ hoạt động có ý nghĩa gần nhất	35
Trễ hạn	Ít nhất một bài tập bắt buộc đã đến hạn mà chưa nộp	30
Tỷ lệ nộp thấp	Tỷ lệ nộp các bài đã đến hạn dưới 50%	20
Hoạt động giảm	Hoạt động giảm ít nhất 50%, với ít nhất 3 quan sát ở cửa sổ trước	15

Điểm tối đa là 100. Các mức: LOW 0–29, MEDIUM 30–59, HIGH 60–79, CRITICAL 80–100. Mọi kết quả kèm đặc trưng, lời giải thích, điều kiện xét, thời điểm đánh giá và chính sách. **Điểm không phải xác suất bỏ học**; đây là công cụ giúp cố vấn ưu tiên.

Vòng đời cảnh báo. Mỗi cặp học viên × khoá học chỉ có một cảnh báo mở. Từ 60 điểm có thể sinh cảnh báo mới; thời gian chờ mặc định là 72 giờ đánh giá. Cảnh báo đang mở được cập nhật mức độ và lý do hiện tại. Cố vấn có thể xác nhận (acknowledge) hoặc giải quyết (resolve); hệ thống lưu thời điểm và người thực hiện. Khi điểm giảm dưới ngưỡng, cảnh báo tự động được giải quyết. Mỗi phiên bản policy ghi tác giả và mốc bắt đầu. Một cập nhật dùng phiên bản dự kiến để chặn ghi đè đồng thời và có hiệu lực tại mốc đánh giá hoàn tất kế tiếp. Gửi lại cùng khóa idempotency và cùng nội dung ghi chú trả về cùng ID. Outbox ghi điểm, phiên bản policy, lý do và chuyển trạng thái để kho lưu được các đánh giá cùng điểm nhưng khác policy hoặc lý do. Trong kịch bản CDC, bốn hoạt động mới đưa learner từ 100 xuống 50 điểm và giải quyết đúng cảnh báo đang mở.

Ví dụ đổi policy. Giả sử phiên bản 1 đã chấm xong mốc T và cố vấn đề xuất nâng ngưỡng từ 60 lên 65 với phiên bản dự kiến là 1. Bản chấm tại T giữ policy 1. Bản chấm ở mốc hoàn tất kế tiếp dùng phiên bản 2 và lưu định danh policy vào lịch sử. Với điểm 62, hai lần đánh giá có cùng điểm nhưng có thể dẫn đến trạng thái cảnh báo khác nhau. Đây là ví dụ về cơ chế phiên

bản. Đường triển khai giữ ngưỡng 60 theo đánh giá ở Mục 5.

4.1.6 BR6 — Tiến độ kỹ năng theo tổ chức

BR6 là ví dụ đầy đủ nhất của pipeline medallion. Telemetry và CDC đi qua silver (chuyển schema, khử định danh), lên gold (`fct_enrollment`, `fct_lesson_progress`, `fct_submission`), rồi tổng hợp thành `mart.org_progress` theo tổ chức × khoá học. Tỷ lệ hoàn thành là tỷ lệ bài học đã xong trung bình của khoá. Mức thành thạo là tỷ lệ đạt test trung bình của các bài nộp đã chấm; học viên chưa có bài chấm được tính là 0.

k-anonymity được bảo vệ ba lớp. (1) Mart chỉ công bố quy mô cho nhóm dưới $k = 5$ người, các tỷ lệ để trống. (2) Cổng chất lượng mức error `mart_org_progress_k_anonymity` làm hỏng lần build nếu vi phạm. (3) Bộ đọc của API xoá các tỷ lệ của dòng bị ẩn, kể cả khi mart lỗi vẫn mang giá trị. Token quản trị viên chỉ được phép đọc các tổ chức trong phạm vi của mình; truy cập ngoài phạm vi trả về 403.

4.1.7 BR7 — Heatmap tương tác video

Theo phiên bản. Sự kiện được quy về phiên bản nội dung có hiệu lực tại thời điểm xảy ra (`gold.dim_content` là SCD2). Vì vậy video quay lại có heatmap riêng cho mỗi phiên bản, và API mặc định chọn phiên bản mới nhất. Bảng `gold.fct_video_second_activity` có grain phiên bản × giây × ngày, đếm play, pause, seek và số người học phân biệt. `mart.video_heatmap` tạo lưới dày cho từng giây trong 28 ngày, có số người học phân biệt chính xác và cường độ chuẩn hoá 0–1. Vị trí vượt quá độ dài video được kẹp về giây cuối.

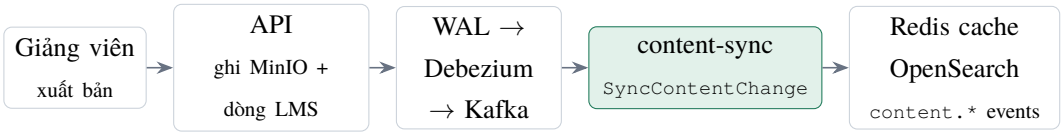
Lớp phủ hôm nay (ADR 0014). Ứng dụng `telemetry-live` chạy ba truy vấn Spark có checkpoint. Truy vấn thứ nhất chuẩn hoá telemetry thô bằng đúng `normalize_rows` của silver. Hai truy vấn còn lại tổng hợp theo giây video (BR7) và điểm tương tác theo ngày (BR8), với watermark 5 phút và khử trùng lặp theo `event_id`. Kết quả ghi vào Redis dưới dạng giá trị tuyệt đối, nên phát lại sẽ ghi đè chứ không cộng thêm. API chỉ ghép lớp phủ cho những ngày sau ảnh chụp của mart, và chỉ cho phiên bản mới nhất. Nếu Redis lỗi, API trả về dữ liệu mart.

4.1.8 BR8 — Đồng bộ nội dung và xu hướng tương tác

4.1.8.1 Đồng bộ cache và chỉ mục tìm kiếm

Một phiên bản nội dung mới là một lần cập nhật dòng `public.course_content`, trỏ tới một đối tượng bất biến mới trong MinIO (`courses/<khoá>/<nội dung>/v<n>.<ext>`). API của giảng viên chỉ ghi đối tượng và dòng dữ liệu, dưới khoá dòng để tránh hai người xuất bản cùng

lúc, chứ không tự đồng bộ. Debezium bắt thay đổi, và dịch vụ `content-sync` chạy use case `SyncContentChange` (Hình 6).



Các bước trong content-sync:
1. Lấy đối tượng 2. Vô hiệu hoá cache Redis 3. Chia đoạn + nhúng vector 4. Dọn phiên bản khác
5. Làm mới mục tìm kiếm 6. Làm ấm cache 7. Phát sự kiện

Hình 6. Đồng bộ nội dung BR8 dựa trên CDC

Idempotent theo trạng thái chỉ mục, không theo thứ tự đến. `sync_policy` chỉ áp dụng phiên bản mới hơn phiên bản đã chỉ mục; bằng nhau là trùng lặp, cũ hơn là lỗi thời. Do đó phát lại CDC, khởi động lại hay backfill đều an toàn. Tìm kiếm chỉ trả các đoạn thuộc phiên bản hiện hành của mỗi nội dung, nên việc dọn dẹp chậm không bao giờ làm lộ văn bản cũ.

Tìm kiếm lai. OpenSearch lưu các đoạn văn bản với chỉ mục BM25 và vector `knn_vector` (HNSW [21]). Kết quả hai phía được hợp nhất bằng reciprocal rank fusion [22] phía client. Vector hiện được sinh bởi một bộ nhúng băm tất định (không cần tải mô hình, không cần GPU), nằm sau cổng `Embedder`; thay bằng mô hình sentence-embedding chỉ cần một adapter mới và một lần reindex. Job Spark `edu batch content-reindex` phân phối *cùng* use case theo từng nhóm nội dung để backfill và ghi `silver.content_chunks`.

4.1.8.2 Xu hướng tương tác

Điểm tương tác của mỗi học viên trong một cửa sổ là tổng có trọng số của các hoạt động (Bảng 18). Mart `mart.engagement_trend` tính 4 cửa sổ tuần (thứ Hai đến Chủ nhật) cho mỗi học viên × khoá học.

Bảng 18. Trọng số hoạt động trong điểm tương tác (`EngagementWeights`)

Nhóm	Động từ (verb)	Trọng số
Đăng nhập	`logged_in`	1,0
Xem bài	`viewed`, `played`, `completed`	1,0
Làm bài	`attempted`, `submitted`	2,0
Thảo luận	`posted`	1,5

Gọi a, b, c là điểm của ba cửa sổ gần nhất theo thứ tự thời gian, và $r = 0{,}30$ là tỷ lệ giảm. Học

viên được đánh dấu *đang giảm* khi

$$a > 0 \ \wedge \ b \leq a\,(1-r) \ \wedge \ c \leq b\,(1-r),$$

tức là điểm giảm ít nhất 30% trong *mỗi* cửa sổ của hai tuần liên tiếp. Lớp phủ "hôm nay" dùng đúng các trọng số này, nên giá trị trong ngày bằng giá trị mà kho dữ liệu sẽ tính vào ngày hôm sau.

4.2 Giao diện người dùng

Pipeline dữ liệu chỉ có giá trị khi kết quả của nó đến được tay người dùng. Mục này trình bày ứng dụng web phục vụ năm nhóm người dùng: người học, giảng viên, cố vấn học tập, quản trị viên tổ chức và đội vận hành nền tảng dữ liệu.

4.2.1 Quy trình thiết kế

Nhóm dùng mockup HTML để thử bố cục theo vai trò, rồi hiện thực giao diện React và TypeScript trong `apps/web`. Instructor đọc kết quả đánh giá có phiên bản, Advisor đọc đặc trưng rủi ro và vòng đời cảnh báo, còn Admin đọc báo cáo theo tổ chức. Các ảnh dưới đây lấy từ bản giao diện đã triển khai. Kịch bản Replay có nhãn dữ liệu mô phỏng.

4.2.2 Hệ thống thiết kế

- **Token thiết kế** cho màu, khoảng cách, bo góc và bóng đổ, có bộ giá trị riêng cho chế độ sáng và tối. Người dùng chuyển chế độ bằng một công tắc; mặc định là chế độ sáng.
- **Bảng màu biểu đồ đã kiểm định**: màu phân loại theo thứ tự cố định và một dải màu xanh đơn sắc cho độ lớn (heatmap), đã kiểm tra khả năng phân biệt với người mù màu. Màu trạng thái (tốt, cảnh báo, nghiêm trọng, nguy hiểm) chỉ dùng cho trạng thái và luôn đi kèm biểu tượng và nhãn chữ, không bao giờ chỉ dựa vào màu.
- **Khả năng tiếp cận**: mọi biểu đồ có tooltip khi rê chuột và một bảng dữ liệu "Xem dạng bảng" thay thế; các nút có nhãn cho trình đọc màn hình; bố cục co giãn tới màn hình điện thoại mà không phải cuộn ngang.
- **Nhãn yêu cầu**: mỗi trang và mỗi khối được gắn nhãn BR tương ứng (có thể tắt). Nhãn này giúp truy vết từ giao diện về yêu cầu khi trình bày.

4.2.3 Kiến trúc frontend

Ứng dụng tổ chức theo *không gian làm việc* (workspace), mỗi nhóm người dùng một workspace, chọn ở thanh bên (Bảng 19). Thanh trên cùng có chỉ báo sự kiện trực tiếp (Server-Sent Events), thông báo của người học và trạng thái đăng nhập.

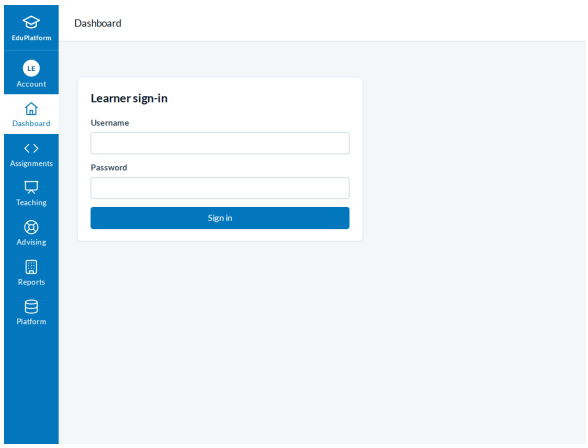
Bảng 19. Workspace, đường dẫn và trang

Workspace	Đường dẫn	Nội dung	BR
Learner	`/learner`	Khoá học, gợi ý, phản hồi gần đây, thông báo	1, 2
Learner	`/learner/code-lab`	Trình soạn mã, phản hồi từng test, các lần nộp	3
Instructor	`/instructor/assessments`	Câu hỏi và chi tiết test case	4
Instructor	`/instructor/videos`	Heatmap theo phiên bản, lớp phủ hôm nay, thời điểm cần xem lại	7
Instructor	`/instructor/engagement`	Trung bình khoá học, học viên đang giảm	8
Instructor	`/instructor/content`	Xuất bản phiên bản, theo dõi đồng bộ, xem trước tìm kiếm	8
Advisor	`/advisor`	Hàng đợi rủi ro, phân rã điểm, lịch sử, cảnh báo	5
Org admin	`/admin`	Tiến độ tổ chức có ẩn nhóm nhỏ, xuất CSV	6
Data platform	`/pipeline`	Bộ đếm luồng, thông lượng, topic, DLQ, silver	–

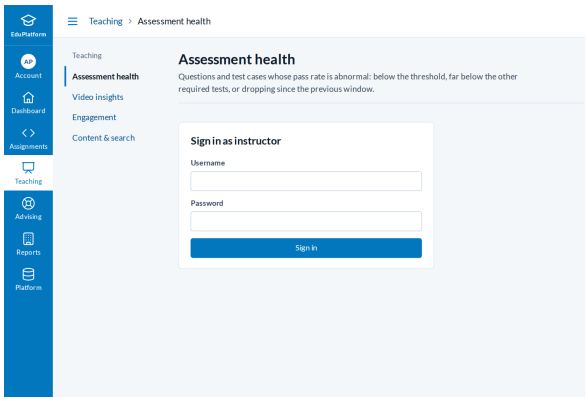
4.2.4 Xác thực và phạm vi dữ liệu

Trong Classroom, người học đăng nhập bằng tài khoản học viên và đọc bài nộp theo quyền sở hữu. Instructor, Advisor và Organisation Admin dùng tài khoản cấp sẵn trong `EDU_STAFF_ACCOUNTS` ở Classroom, Persistent và Replay. Lệnh cấp tài khoản nhận mật khẩu từ lời nhắc ẩn và xuất hash Argon2id [23]. Máy chủ đặt cookie HttpOnly, SameSite=Strict và bật Secure trên HTTPS. Mỗi vai trò có một phiên tám giờ trên trình duyệt, độc lập với vai trò khác và Student. Đăng xuất thu hồi phiên tương ứng. Khởi động lại API kết thúc các phiên staff trong tiến trình.

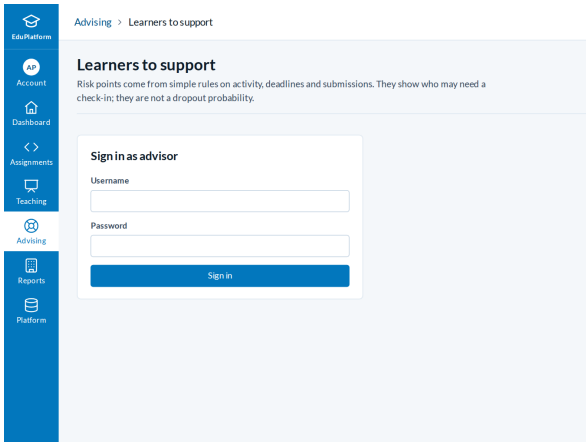
Ứng dụng gửi `X-Staff-Role` để máy chủ chọn cookie phiên. Máy chủ kiểm tra vai trò, phạm vi khóa học hoặc tổ chức và cùng origin đối với thao tác ghi bằng cookie. Script có thể dùng Bearer token khai báo trong `EDU_BR45_AUTH`. Khi Trino chưa sẵn sàng, Admin vẫn đăng nhập và trang báo cáo hiển thị lỗi tải dữ liệu.



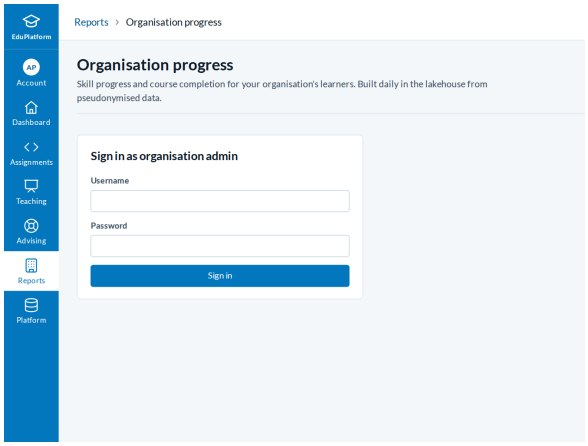
(a) Student, Classroom



(b) Instructor, Persistent



(c) Advisor, Persistent



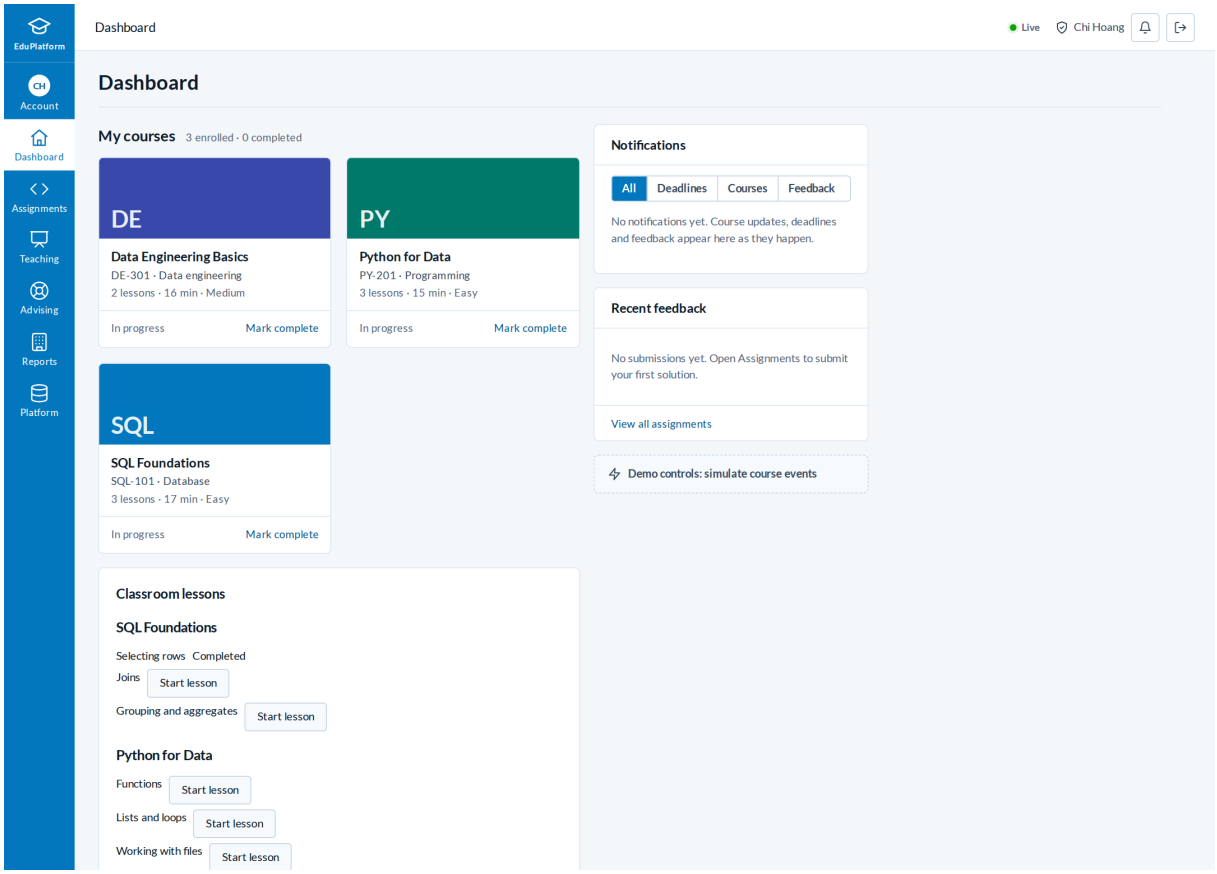
(d) Organisation Admin, Persistent

Hình 7. Form đăng nhập theo vai trò trước khi nhập mật khẩu

4.2.5 Các màn hình chính

Các fixture là dữ liệu tổng hợp. Classroom lưu bài học và bài nộp vào PostgreSQL. Ảnh Persistent BR4/BR5 chụp sau khi kết quả đã đi qua CDC và worker chạy thật. Ảnh BR6–BR8 dùng các mart Trino và dịch vụ nội dung của stack báo cáo. Pipeline Health hiển thị luồng sự kiện demo của ứng dụng.

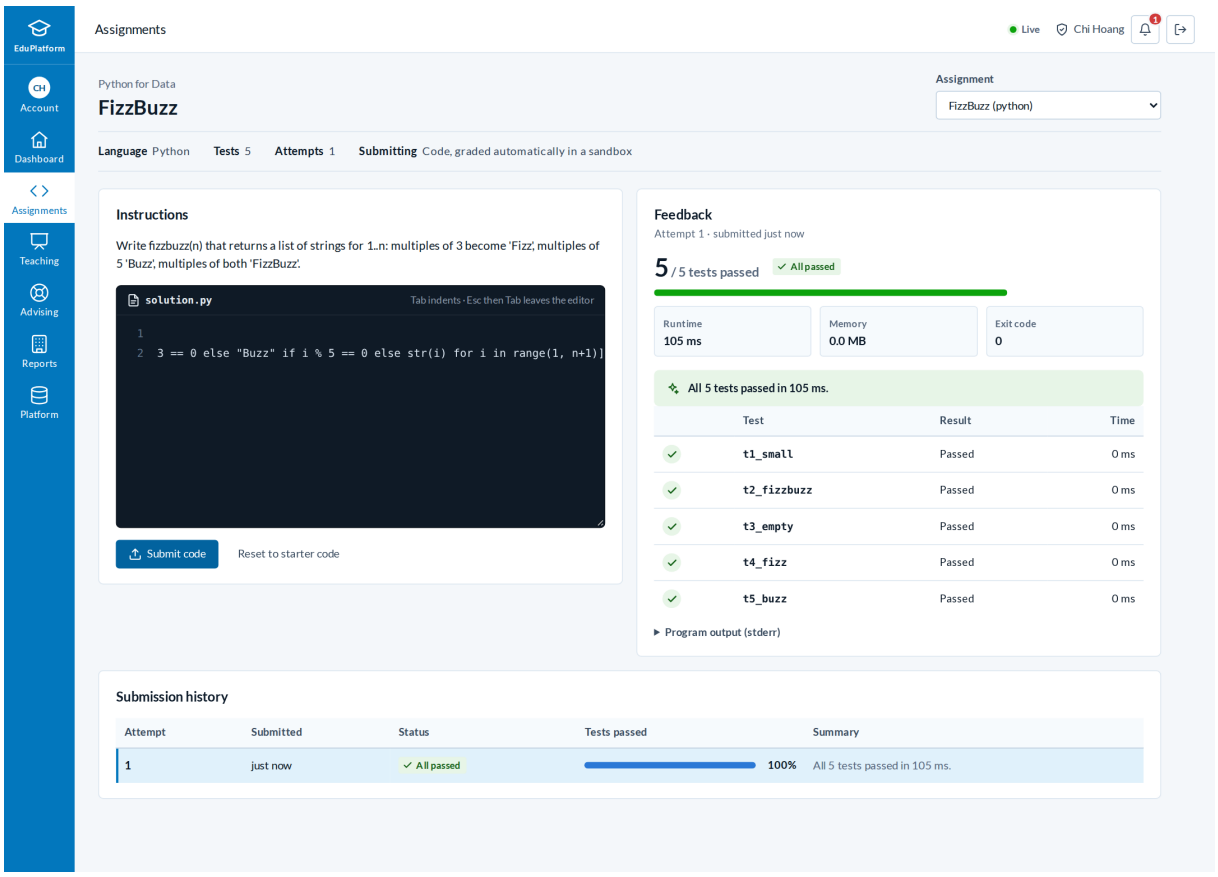
Giảng viên xem thay đổi sau sửa điểm ở Hình 10a và 10b. Cố vấn theo dõi cùng cảnh báo khi người học phục hồi ở Hình 11a và 11b. Quản trị viên xem ngưỡng công bố ở Hình 16a và 16b, còn phạm vi phiên Acme được minh họa ở Hình 17.



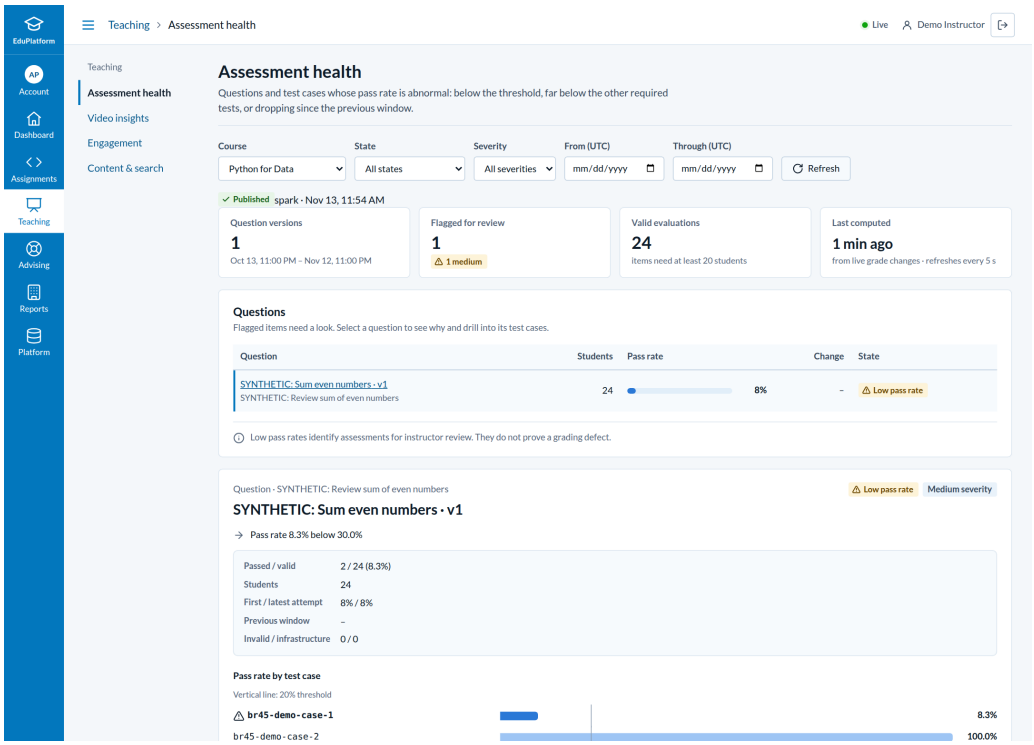
Hình 8. Learner Home trong Classroom. Người học Chi thấy khoá học, thông báo và tiến độ bài học

4.2.6 Kiểm thử giao diện

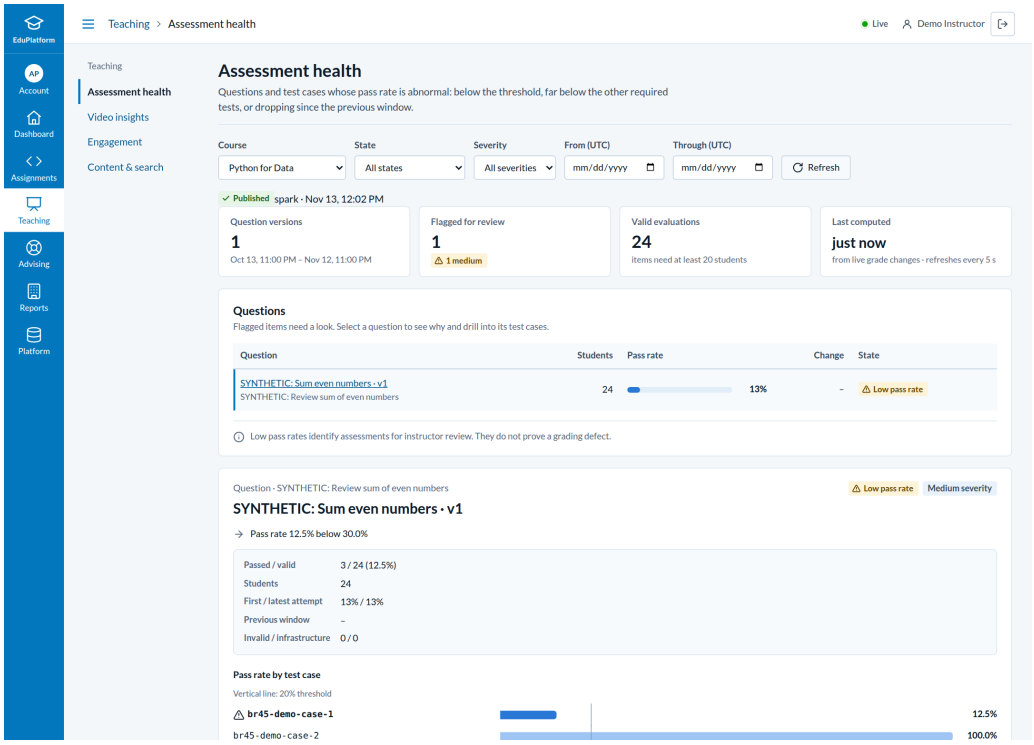
TypeScript ở chế độ nghiêm ngặt và bản build production đã qua kiểm tra. Năm lượt Playwright sau thay đổi auth kiểm tra phiên staff trong ba chế độ, Replay dùng chung kịch bản và bài nộp Code Lab trong Classroom. Các phép kiểm thử API xác nhận hết hạn, đăng xuất độc lập, kiểm tra quyền sở hữu Student, phạm vi staff và tương thích Bearer. Ảnh BR4/BR5 bổ sung minh chứng cho những thay đổi xuất hiện trên dashboard sau CDC. Bảng ánh xạ ảnh cũ, nguồn chụp và trạng thái được lưu cùng nguồn LaTeX.



Hình 9. Code Lab trong Classroom. Lời giải FizzBuzz nhận phản hồi đã chấm và kết quả theo test

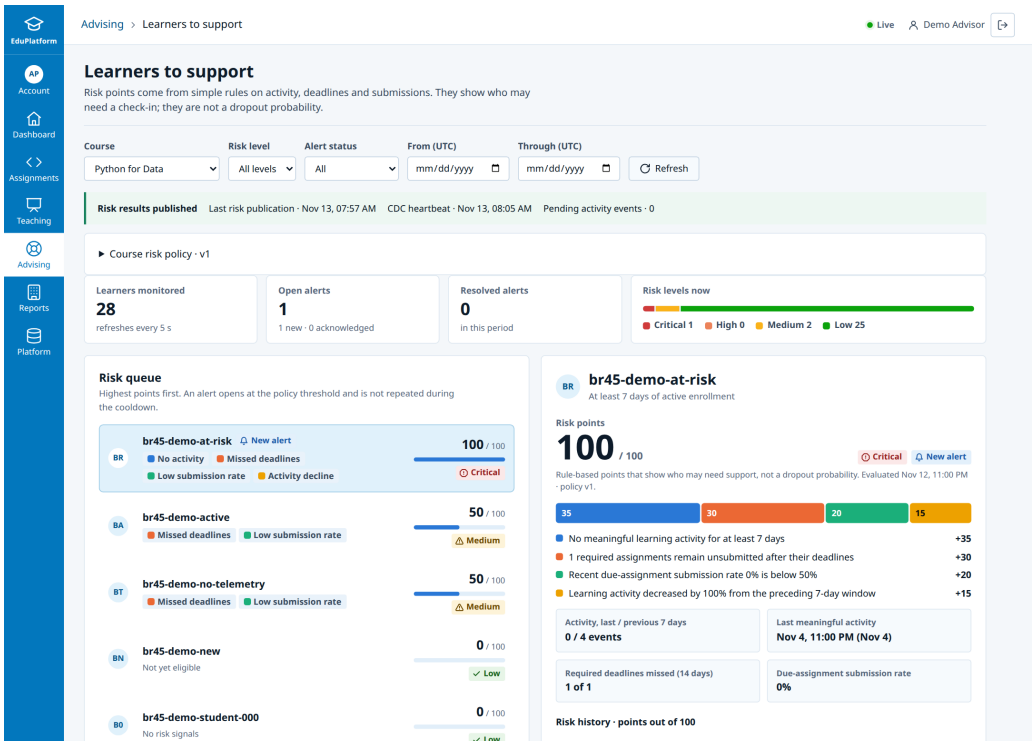


(a) Trạng thái cơ sở có 2/24 lượt đạt và một test cần xem xét. Delta version 16

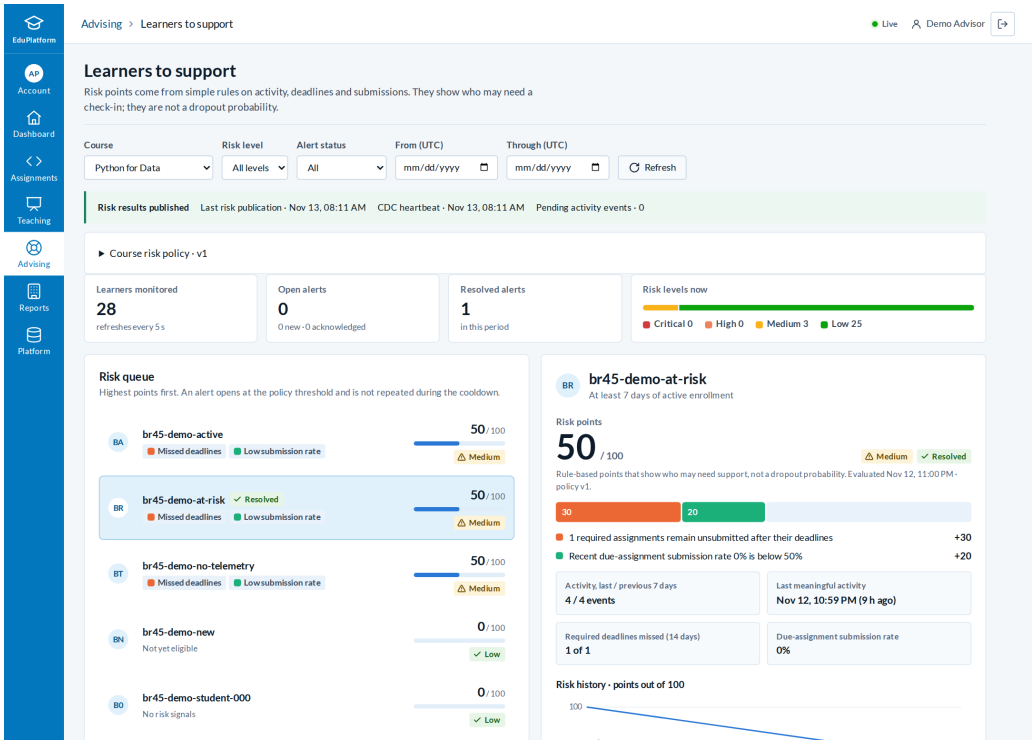


(b) Sau sửa điểm qua CDC, worker Spark/Delta công bố 3/24 lượt đạt. Delta version 17

Hình 10. Assessment Health trong Persistent với fixture Python for Data

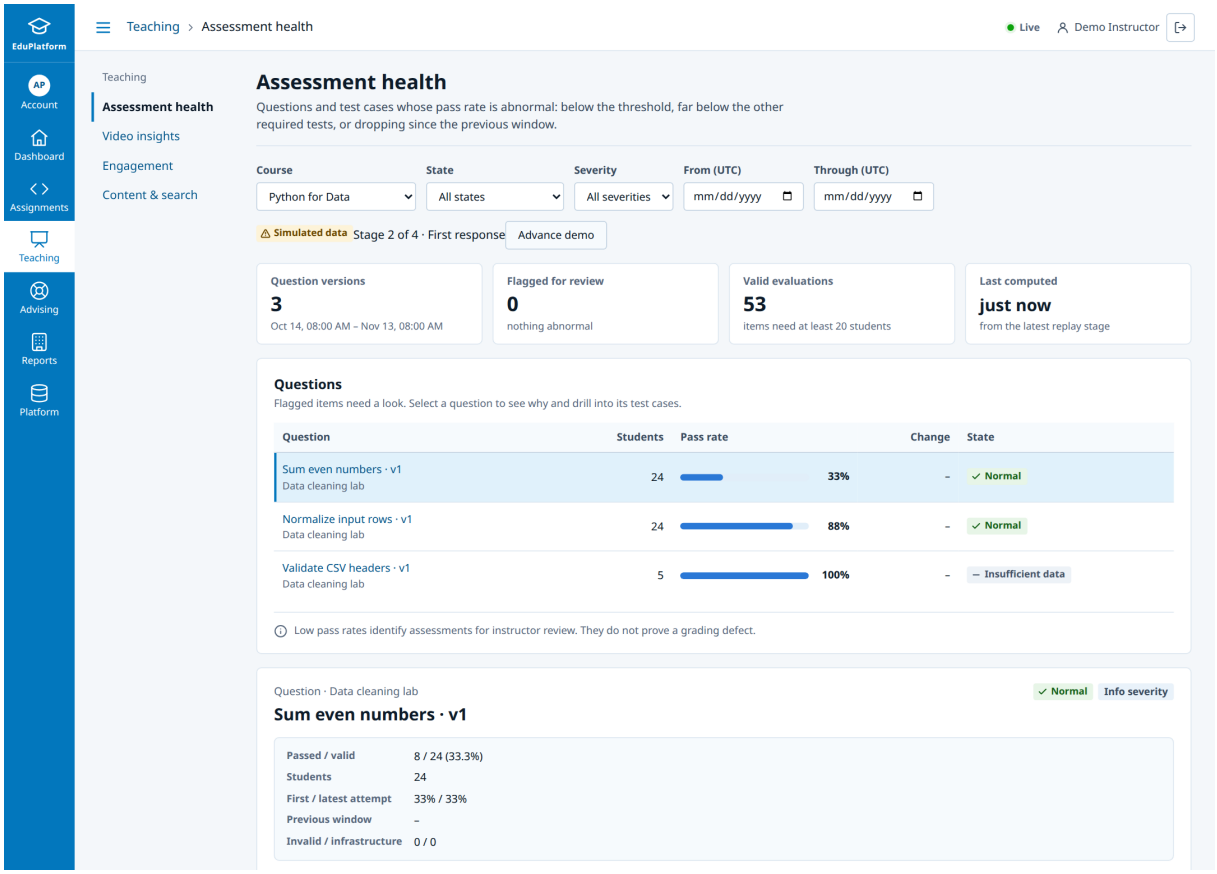


(a) Học viên mẫu có 100 điểm và một cảnh báo mới

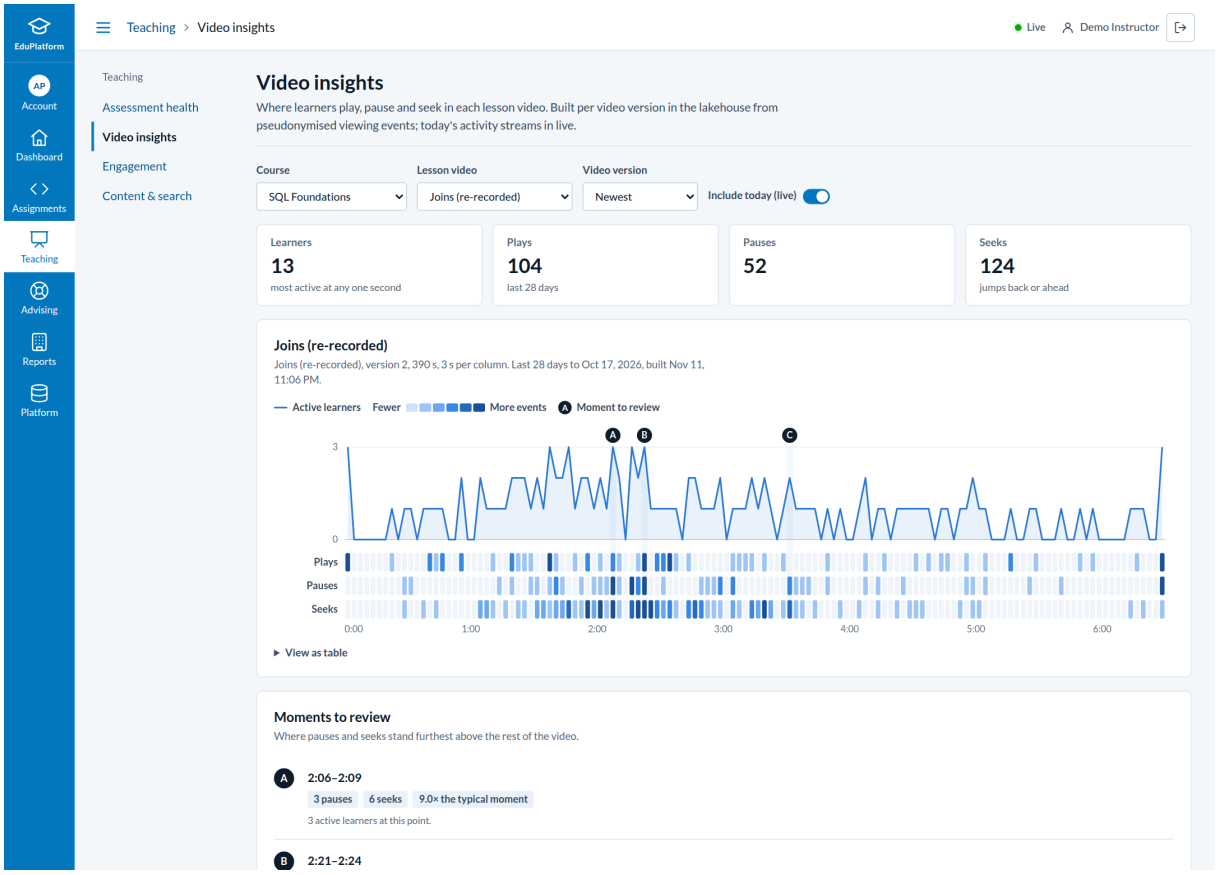


(b) Sau bốn hoạt động mới, điểm giảm còn 50 và cảnh báo được giải quyết

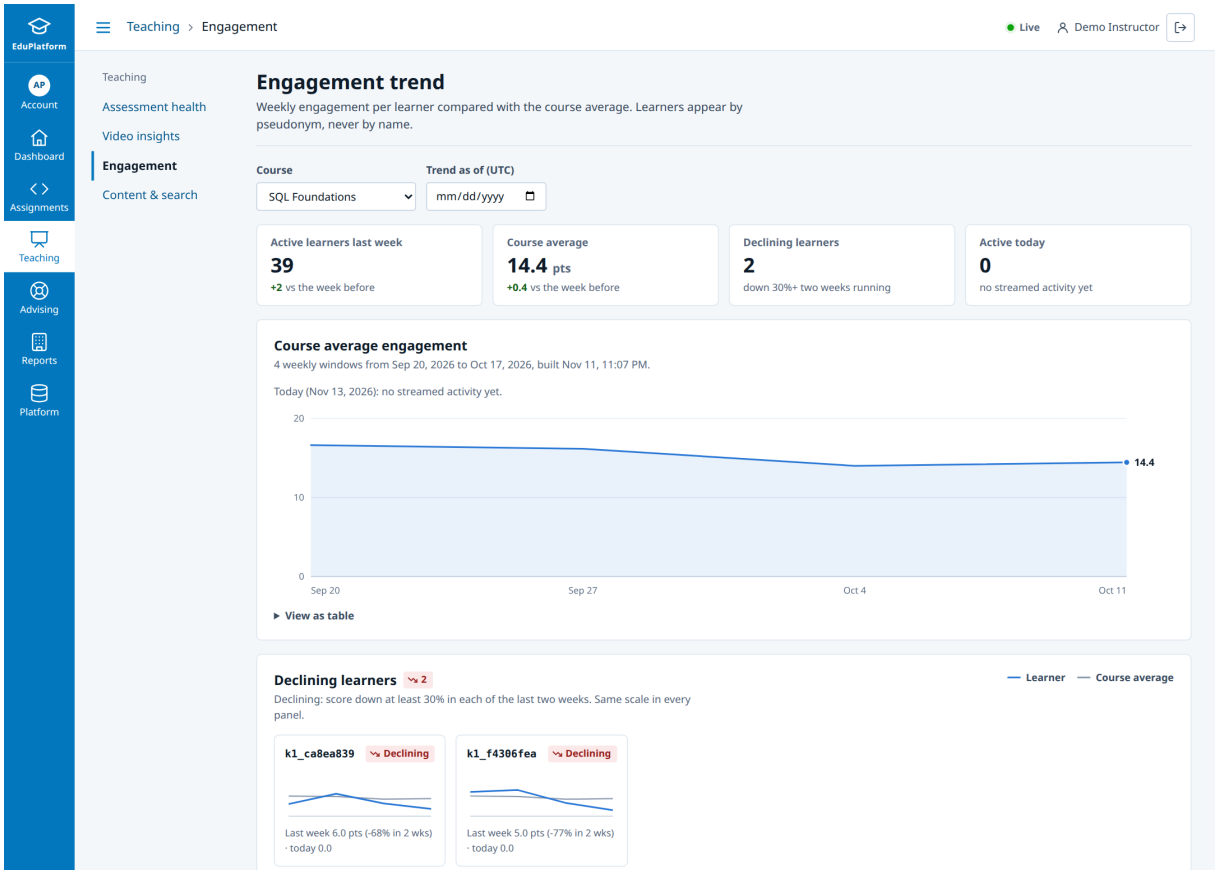
Hình 11. Vòng đời cảnh báo trong Learners to support, chế độ Persistent



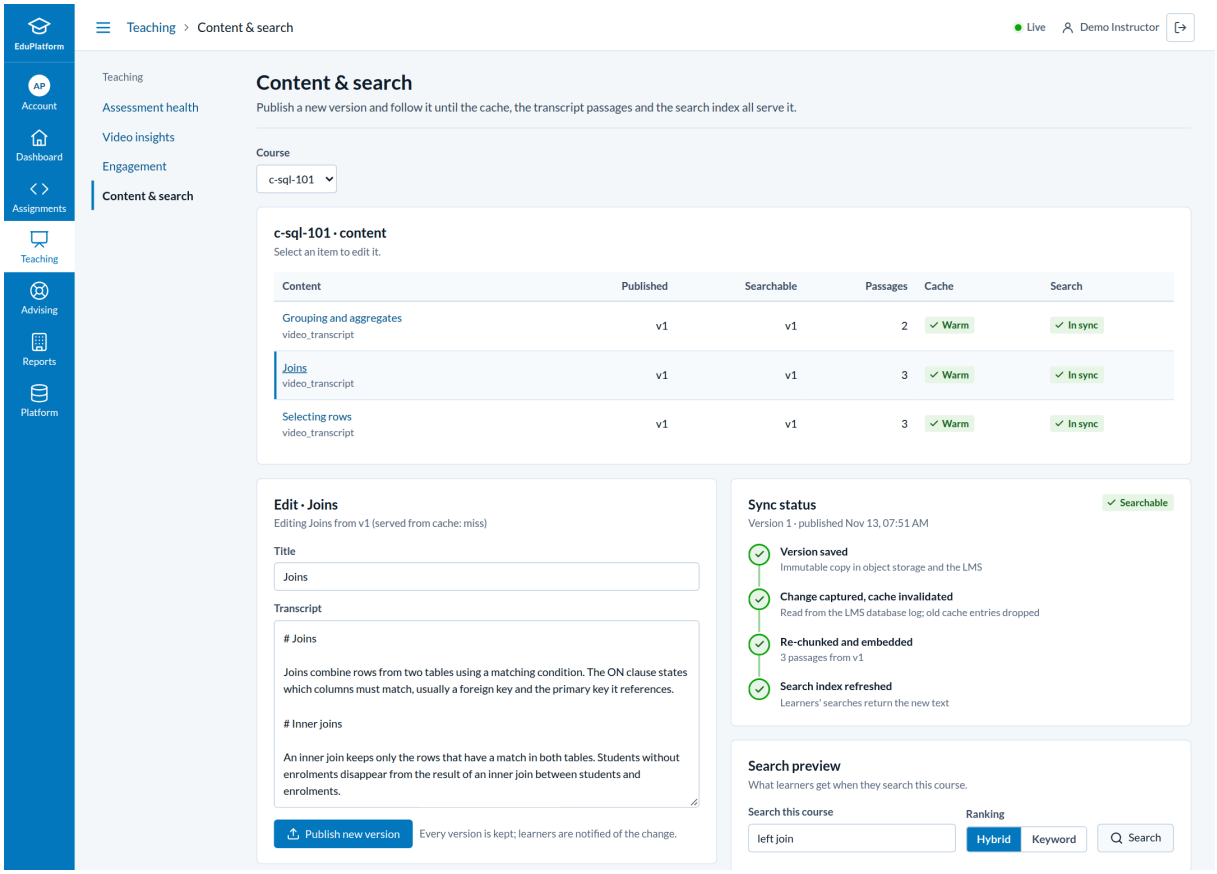
Hình 12. Assessment Health trong Replay, có nhãn Simulated data



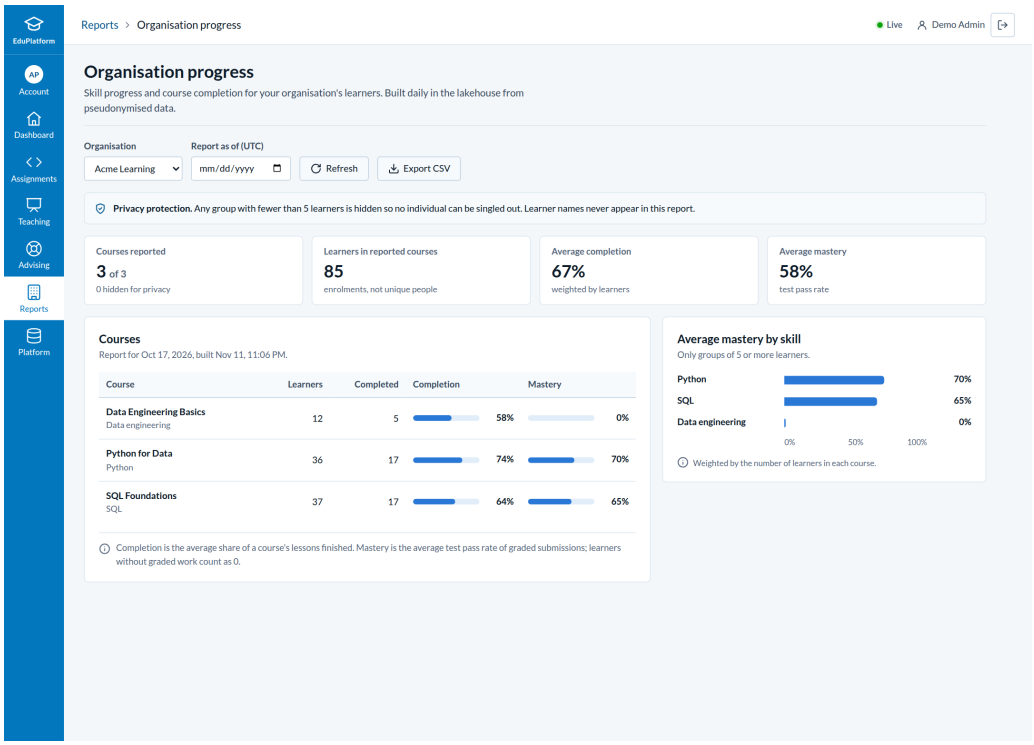
Hình 13. Video Insights trong Persistent. Mart Trino hiển thị phiên bản 2 của video Joins đến 17/10/2026



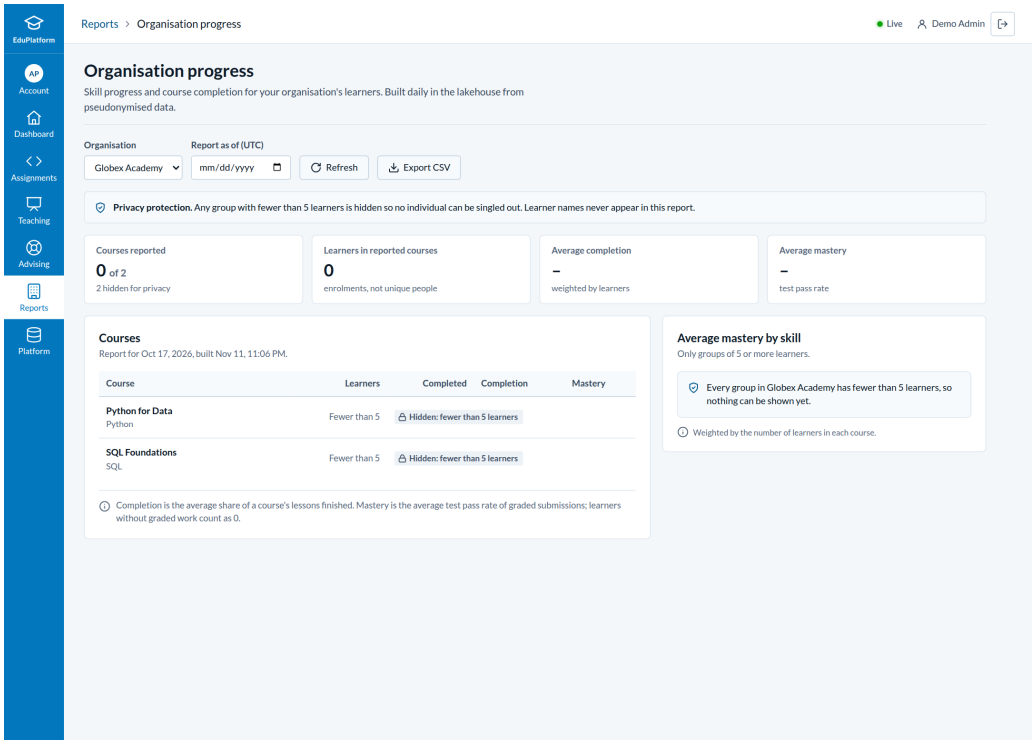
Hình 14. Engagement trong Persistent. Mart Trino hiển thị xu hướng theo tuần của SQL Foundations



Hình 15. Content & Search trong Persistent. Transcript Joins đã đồng bộ và có thể tìm kiếm

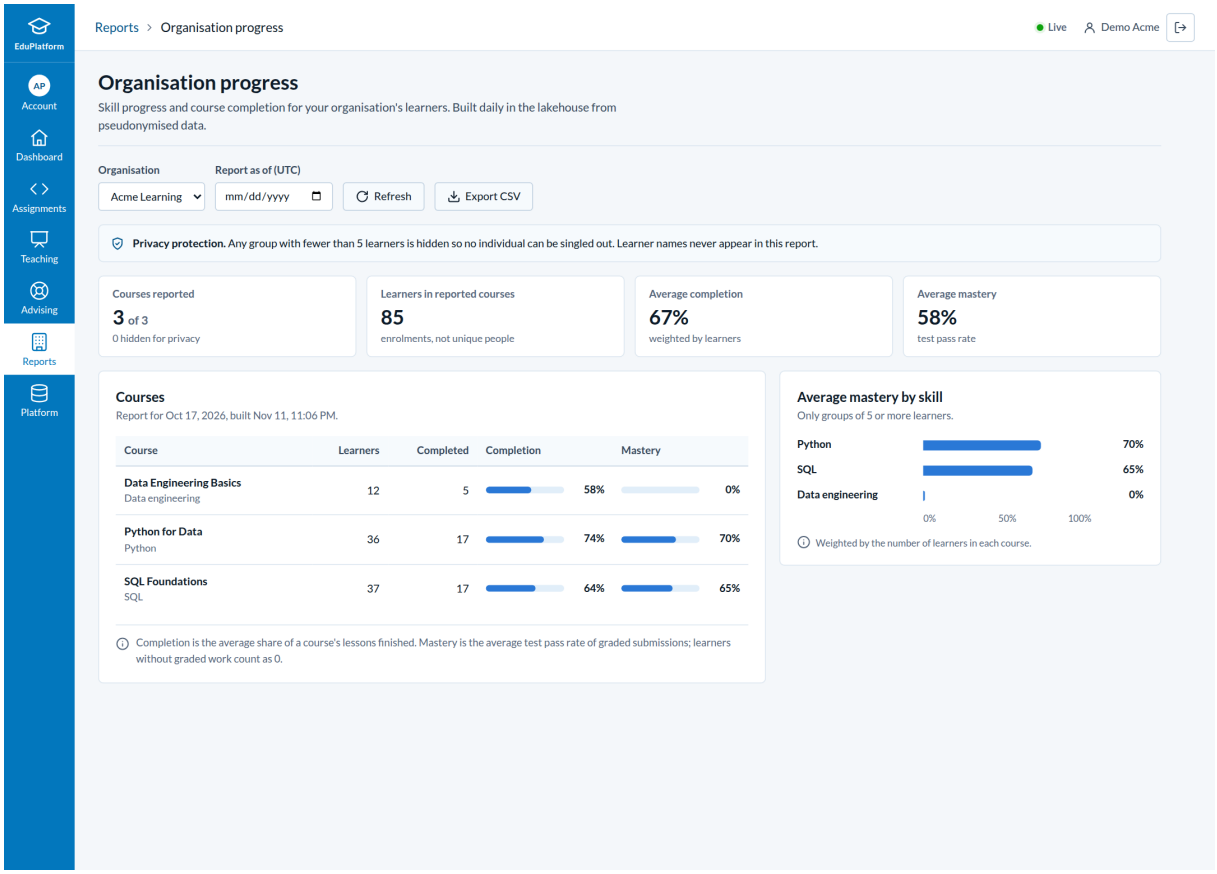


(a) Acme hiển thị số liệu của nhóm đủ điều kiện

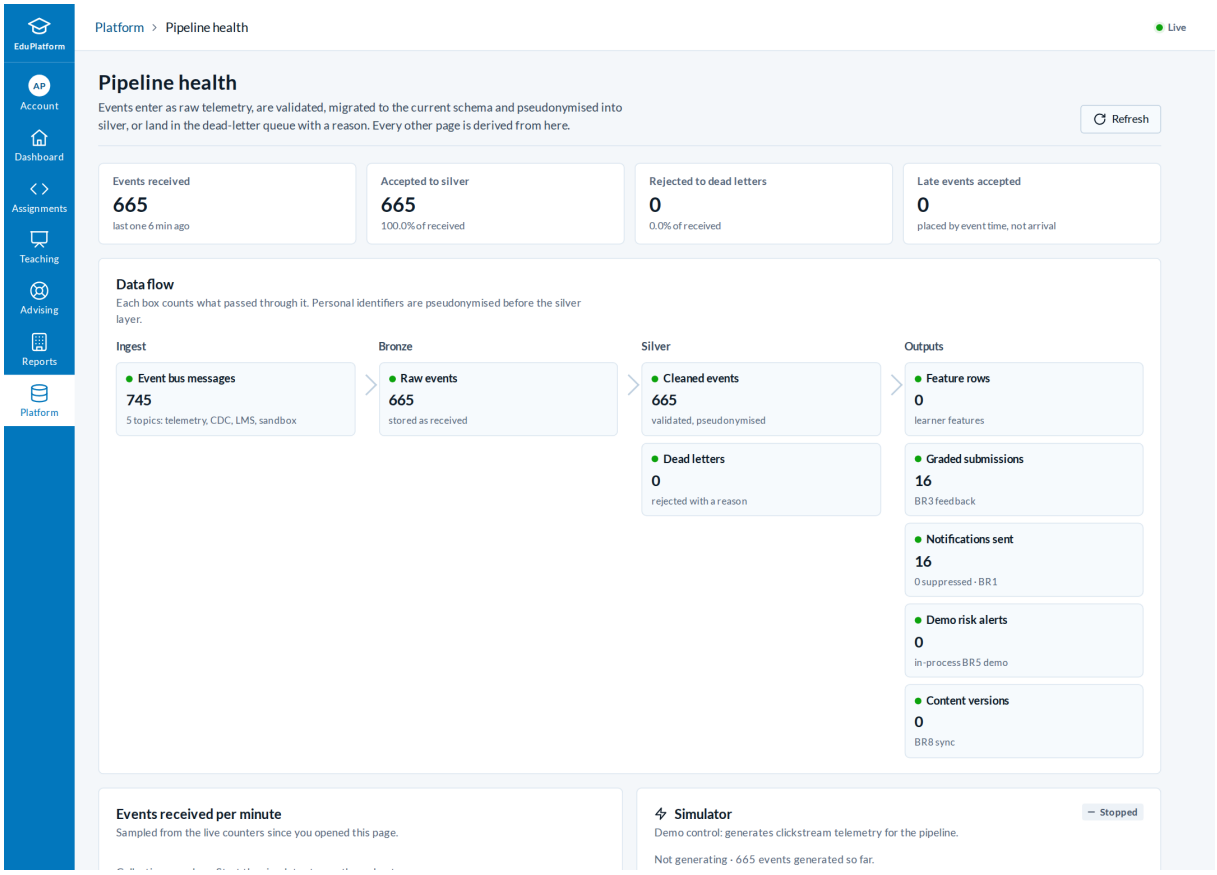


(b) Globex ẩn tỷ lệ và số hoàn thành của nhóm dưới 5 học viên

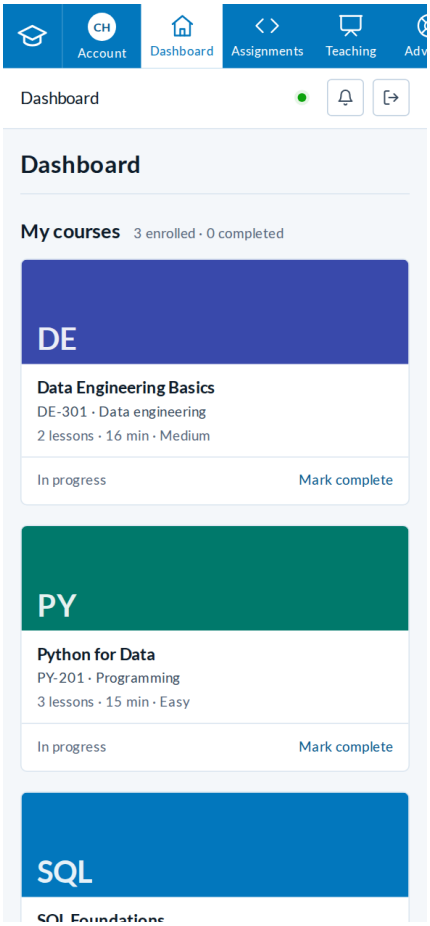
Hình 16. Organisation Progress trong Persistent với snapshot kho ngày 17/10/2026



Hình 17. Phiên Admin chỉ có phạm vi Acme. Yêu cầu kiểm chứng đến API Globex trả 403



Hình 18. Pipeline Health. Các bộ đếm và topic của luồng demo



Hình 19. Learner Home trong Classroom ở chiều rộng 390 px

5 Kiểm thử và đánh giá

Đánh giá gồm ba tiêu chí theo CO5173: tính đúng sau xử lý, hiệu năng và hiệu quả khai thác dữ liệu. Tính đúng được đối chiếu với oracle và trạng thái quan sát qua API. Hiệu năng dùng độ trễ từ mốc đầu đến kết quả đã lưu hoặc API và thông lượng theo số sự kiện duy nhất. Hiệu quả được kiểm tra bằng thao tác nghiệp vụ trong ứng dụng và bằng nhãn rút học ngoại tuyến của OULAD. Nhật ký gốc và cách chạy nằm ở Phụ lục A.

5.1 Chiến lược kiểm thử

Bảng 20. Các tầng kiểm thử

Tầng	Số lượng	Phạm vi
Python/PostgreSQL	379	Đợt hồi quy sau auth gồm đơn vị, tích hợp BR4/BR5 và hồi quy API BR6–BR8.
Spark/Delta trực tiếp	2	Snapshot khớp oracle với kết quả thiếu, sai, lỗi hạ tầng và nhiều phiên bản. Spark phục hồi checkpoint, khử trùng và rút lại hoạt động.
Đầu cuối CDC	6 kiểm tra	Sửa điểm qua API, phát lại, UPDATE/DELETE hoạt động và phục hồi checkpoint. Phép thử riêng chèn lỗi publish xác nhận mart cũ vẫn phục vụ.
Trình duyệt	5	Sau cập nhật auth, Playwright đạt ở ba chế độ, gồm đăng nhập staff, Code Lab Classroom và trạng thái Replay dùng chung. Các lượt chấp nhận CDC và BR6–BR8 có nhật ký riêng.

Ngoài ra, mã Python qua `ruff` và `mypy`; kho dữ liệu chạy cả 10 cổng chất lượng sau mỗi lần build.

5.2 Truy vết yêu cầu và kết quả sử dụng

Bảng 21. Từ yêu cầu và dữ liệu đến kết quả, minh chứng và giới hạn

BR và người dùng	Dữ liệu	Kết quả sử dụng	Minh chứng và giới hạn
BR1 Người học	Sự kiện bài tập, hạn nộp, học liệu	Xem thông báo phù hợp với khóa học	Giao diện demo và kiểm thử luật. Tích hợp từ dịch vụ Kafka đến hộp thông báo cần nghiệm thu.
BR2 Người học	Ghi danh, lịch sử học, đặc trưng dài hạn	Xem gợi ý kèm lý do	Luật gợi ý và bảng đặc trưng lô. Chưa có nhánh phiên qua feature store.
BR3 Người học	Mã nộp, stderr, diff, kết quả từng test	Chẩn đoán lỗi và nộp lại ở Code Lab	Classroom giữ cùng ID đến năm kết quả test và facts BR4. Luồng worker Kafka riêng chưa được nối đầu cuối với giao diện này.
BR4 Giảng viên	Bài nộp và kết quả chấm theo phiên bản	Khoanh vùng test bất thường và xem tác động sửa điểm	$2/24$ thành $3/24$ qua CDC. Oracle, kiểm tra bản công bố và lỗi publish. Bốn trường hợp gắn nhãn là fixture tổng hợp.
BR5 Cố vấn	Hoạt động, ghi danh, deadline và kết quả nộp	Ưu tiên can thiệp và theo dõi phục hồi	100 xuống 50 điểm, một cảnh báo được giải quyết sau phát lại. OULAD đo khả năng dự báo hồi cứu.
BR6 Quản trị viên	Ghi danh và kết quả đã khử định danh	So sánh tiến độ tổ chức được phép	Acme và Globex minh họa ngưỡng $k = 5$. Nhóm nhỏ ẩn tỷ lệ và số hoàn thành. Truy cập ngoài phạm vi trả 403.
BR7 Giảng viên	Tương tác theo giây, phiên bản video	Xác định đoạn cần xem lại	Hai phiên bản video có heatmap riêng. Chỉ số *seek* hiện gộp cả tua tới và tua lùi.
BR8 Giảng viên	Phiên bản transcript, CDC và sự kiện tương tác	Xuất bản, theo dõi tìm kiếm và xem xu hướng giảm	Các bài đo đồng bộ nội dung và lớp phủ ở Mục 5. Embedding băm chủ yếu phản ánh từ vựng.

Trong bốn kịch bản BR4 có nhãn, oracle và phân loại khớp: test đáng ngờ, câu hỏi tỷ lệ thấp,

đề bình thường và mẫu không đủ. Hai dương tính và hai âm tính đều đúng trên fixture này. Bài sửa điểm, bài đọc snapshot, bài restart cửa sổ và bài xác thực phạm vi là các minh họa kỹ thuật riêng. Phép thử Spark riêng kiểm tra bản ghi thiếu hoặc không hợp lệ, lỗi hạ tầng và phiên bản test. Khi chèn lỗi trong transaction publish, API giữ mart cũ. Diễn tập khôi phục so checksum các mart, gửi lại ID cũ, thêm hoạt động mới và xác nhận không trùng cảnh báo. Job kế thừa cần đợi lease hết hạn trước khi worker tiếp quản.

Trong diễn tập backfill, nhóm tách cơ sở dữ liệu, connector, topic, nhóm tiêu thụ và checkpoint khỏi đường cũ. Đối chiếu số dòng với nguồn và hai cửa sổ với oracle trước khi chuyển API cùng writer. Sau chuyển đổi, sự kiện kiểm tra chỉ hạ cánh ở đích mới. Giữ deployment cũ để quay lui. Lệnh retention mặc định trả bản xem trước và kiểm tra công việc hoặc dữ liệu chờ trước khi xóa.

5.3 Môi trường và phương pháp đo

Các nhật ký engine BR4/BR5 ngày 12/11/2026 dùng fixture tổng hợp và revision `3cfd70d`. Máy Linux x86-64 có 6 CPU logic và khoảng 18,4 GiB RAM. Cấu hình đối chiếu ghi 2 CPU và 2 GiB cho các engine. BR4 dùng PostgreSQL 16.15 và Spark 3.5.5. BR5 dùng Spark 3.5.1 và Flink 1.19.3, publish mỗi 5 giây và Flink checkpoint mỗi 10 giây. Mỗi cỡ có một lượt làm nóng và ba lượt đo. Mỗi tình huống đầu cuối có năm lượt đo. Kết quả chính đạt 36/36 lượt xử lý, 30/30 lượt đầu cuối và 2/2 kiểm tra restart. Bảng ghi trung vị và khoảng nhỏ nhất đến lớn nhất. Phép đo BR6–BR8 ngày 02/11 dùng VM Colima 6 CPU, 10 GB, Spark 3.5.5, Delta 3.3.2, Trino 478 và OpenSearch 2.17.1.

5.4 Kết quả hiệu năng

5.4.1 BR4: PostgreSQL SQL và Spark SQL

Đồng hồ chặng xử lý chạy từ lúc gọi tổng hợp đến khi ghi mart. Spark đọc cùng dữ liệu đã chuẩn hóa qua JDBC. Đường đo này gồm tổng hợp cửa sổ hiện tại và kỳ trước và ghi PostgreSQL. Đường triển khai Spark/Delta còn phải hạ cánh Delta, phân tích cohort và ghi facts, dimensions theo từng bản công bố. Kịch bản sửa điểm của harness đo từ sau commit nguồn đến khi API đọc được 3 lượt đạt, với harness điều phối lần tính.

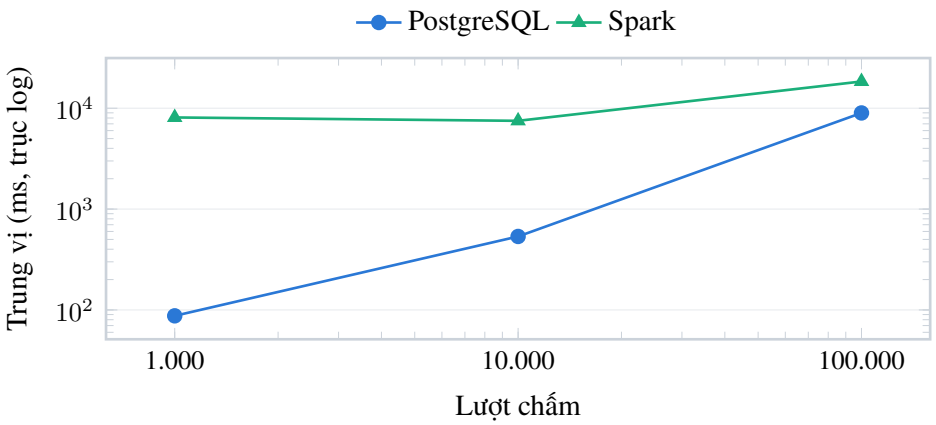
Ở cả ba cỡ PostgreSQL có trung vị thấp hơn trong chặng đo BR4. Chi phí khởi động và điều phối Spark ảnh hưởng rõ khi cỡ nhỏ. Nhóm giữ PostgreSQL làm phương án phục vụ thay thế. Đường Spark/Delta hỗ trợ truy vết theo phiên bản và đọc lại snapshot cho điều tra lỗi chấm.

Bảng 22. BR4, xử lý và ghi mart qua harness (ms)

Lượt chấm	PostgreSQL	Spark	Đạt
1.000	87,40 (78,26–127,08)	8.098,51 (6.809,85–9.066,63)	3/3 mỗi engine
10.000	535,17 (479,25–556,51)	7.500,32 (3.570,18–10.949,06)	3/3 mỗi engine
100.000	8.976,89 (8.547,57–13.123,40)	18.412,37 (17.413,07–27.804,69)	3/3 mỗi engine

Bảng 23. BR4, sửa điểm qua CDC với harness điều phối (ms)

Kịch bản	Engine	Trung vị (min–max)	Đạt
Sửa điểm	PostgreSQL	1.138,49 (856,35–2.128,14)	5/5
Sửa điểm	Spark	3.995,36 (3.551,52–5.681,41)	5/5



Hình 20. BR4, tổng hợp và ghi mart qua harness

5.4.2 BR4: sửa điểm qua worker triển khai

Hai project Compose cô lập dùng cùng fixture 24 học viên, mốc 16:00 UTC ngày 12/11/2026 và cùng cấu hình CDC. Worker có giới hạn 2 CPU và 2.304 MiB. Hai đường dùng PostgreSQL 16.15. Đường Spark 3.5.5 và Delta 3.3.2 lưu snapshot và bản công bố trên MinIO. Nhật ký ngày 13/11 ghi bản mã API `eab34a8` kèm thay đổi auth, bản mã image Spark `1ac28e1`, image ID và giới hạn của từng dịch vụ.

Đồng hồ bắt đầu ngay sau commit sửa một lần chấm từ trượt sang đạt. Client thăm dò API mỗi 250 ms đến khi bản công bố thành công trả 3/24, đúng engine, có publication và source revision mới. Đường Spark phải có Delta version tăng. Mỗi engine có một lượt làm nóng và năm lượt đo. Sau mỗi lượt, client trả fixture về 2/24 và đợi công bố trước lần sửa tiếp theo. Bảng 24 tính độ trễ trên các lượt đạt, với giới hạn chờ 480 giây và nhật ký riêng cho từng lỗi.

Bảng 24. BR4 triển khai, sửa điểm từ 2/24 lên 3/24 (giây)

Worker	Làm nóng	Đạt	Trung vị (min–max)	Lỗi / timeout
PostgreSQL SQL	4,14	5/5	4,29 (1,64–6,55)	0/0
Spark SQL/Delta	403,13	5/5	110,11 (48,26–237,46)	0/0

Phép đo này bao gồm PostgreSQL WAL, Debezium, Kafka, worker và lần đọc API. Đường Spark còn chờ hạ cánh snapshot Delta và ghi facts, dimensions trước khi thay mart. Trong các lượt đã quan sát, PostgreSQL công bố sớm hơn. Máy dùng chung có tải nền thay đổi, gồm thời gian khôi phục kho dữ liệu cho ảnh. Vì vậy độ chênh quan sát được gộp cả chi phí của đường xử lý và thời gian tranh chấp tài nguyên. Nhật ký giữ riêng lượt PostgreSQL bị chồng thao tác trong lúc phục hồi dịch vụ.

5.4.3 BR5: Spark Structured Streaming và Flink

Chặng xử lý bắt đầu khi gửi sự kiện vào Kafka và kết thúc khi sink PostgreSQL có đúng cửa sổ, đồng thời offsets xác nhận đã tiêu thụ. Thông lượng lấy số sự kiện duy nhất chia cho thời gian quan sát. Đầu vào còn có khoảng 1% bản sao và 16 sự kiện đồng hồ ở mỗi lượt. Kiểm tra đúng đắn gồm trùng, sai thứ tự, biên nửa mở và rút đóng góp. Trong kịch bản cảnh báo và phục hồi, đồng hồ bắt đầu sau commit hoạt động nguồn và dừng khi API trả điểm và trạng thái cảnh báo đúng. Hai engine dùng chung bộ tính và lưu điểm sau sink benchmark. Cả hai khôi phục đúng tổng cửa sổ và định danh sau restart.

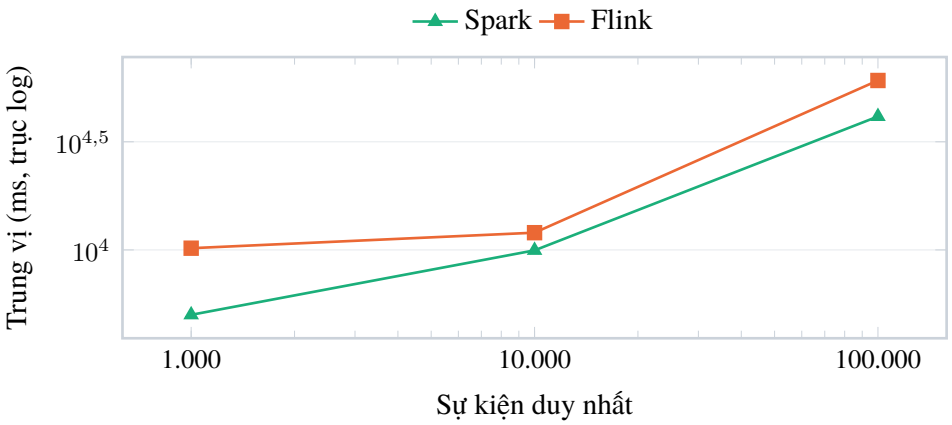
Ở ba cỡ đã đo, trung vị chặng Kafka đến cửa sổ của Spark thấp hơn Flink trong cấu hình này. Một lượt nghiệm thu riêng trên đường Spark triển khai ghi nhận 20,97 giây từ CDC đến cảnh báo và 35,46 giây từ restart đến phục hồi. Hai thời gian nghiệm thu mô tả đường chạy thực tế và

Bảng 25. BR5, Kafka đến cửa sổ đã lưu (ms)

Sự kiện	Engine	Trung vị (min–max)	Sự kiện/s	Đạt
1.000	Spark	5.010,15 (4.960,66–15.918,41)	200	3/3
1.000	Flink	10.184,82 (10.109,44–10.759,95)	98	3/3
10.000	Spark	9.953,72 (4.959,91–10.398,19)	1.005	3/3
10.000	Flink	12.014,08 (10.983,26–15.715,66)	832	3/3
100.000	Spark	41.403,15 (25.051,28–43.425,91)	2.415	3/3
100.000	Flink	60.746,89 (29.868,94–62.898,08)	1.646	3/3

Bảng 26. BR5, commit nguồn đến API Advisor trong benchmark (ms)

Kịch bản	Engine	Trung vị (min–max)	Đạt
Cảnh báo	Spark	13.063,42 (7.571,60–21.234,98)	5/5
Cảnh báo	Flink	14.252,88 (12.777,47–23.756,56)	5/5
Phục hồi	Spark	10.234,79 (5.710,18–16.204,69)	5/5
Phục hồi	Flink	15.203,97 (11.496,49–25.871,05)	5/5



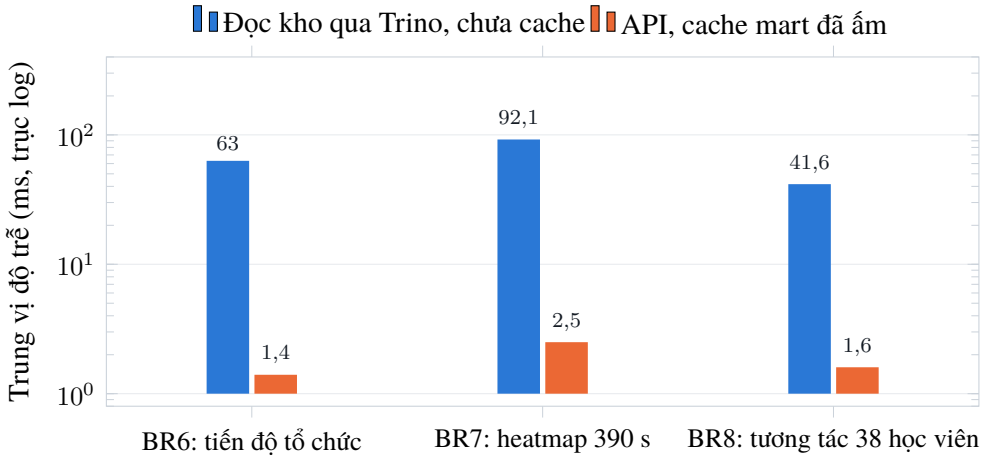
Hình 21. BR5, Kafka đến cửa sổ đã lưu

không được gộp vào phân phối benchmark.

5.4.3.1 Số liệu lịch sử tháng 10.

Ở 100.000 lượt chấm, trung vị BR4 là 3,90 giây cho PostgreSQL SQL và 1,95 giây cho tính toán Python trên dữ liệu đã chuẩn hóa. Ở 100.000 sự kiện, trung vị BR5 là 15,17 giây từ Kafka đã chuẩn hóa đến khi lưu cửa sổ Spark. Các số này dùng ranh giới của phép đo cũ. CSV gốc nằm trong `data/`.

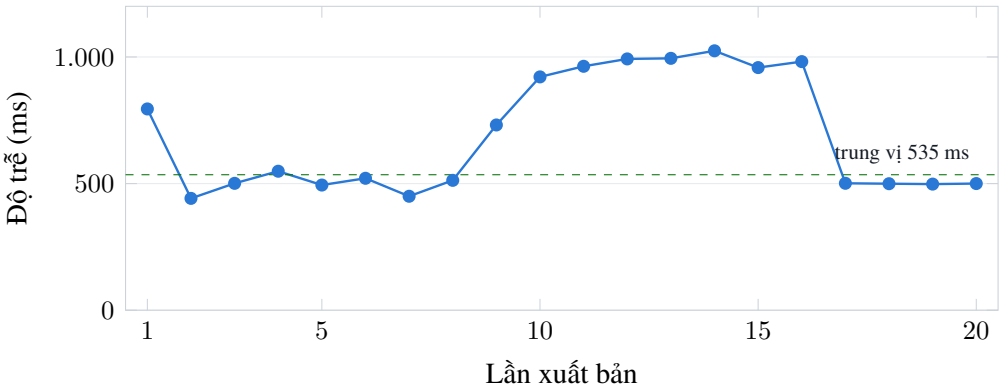
5.4.4 BR6–BR8: phục vụ mart



Hình 22. Độ trễ phục vụ các mart BR6–BR8 (30 request mỗi trường hợp)

Bách phân vị 90 khi đọc thẳng qua Trino là 92, 136 và 59 ms cho BR6, BR7, BR8; khi có cache là 1,8, 2,8 và 2,0 ms. Vì mart chỉ thay đổi mỗi ngày còn dashboard thăm dò mỗi 5 giây, bộ đọc có cache TTL 60 giây (`CachedMartReader`). Cache này còn sửa được một lỗi thật: Trino bị hết bộ nhớ khi dashboard thăm dò liên tục mà không có cache.

5.4.5 BR8: từ xuất bản đến khi tìm kiếm được



Hình 23. BR8: độ trễ từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp

API nhận yêu cầu xuất bản trong 16,9 ms (trung vị). Toàn bộ đường đi PostgreSQL WAL → Debezium → Kafka → content-sync → tìm kiếm được có trung vị **535 ms**, p90 992 ms, lớn nhất 1.024 ms. Bản thân bước đồng bộ chỉ mất 25–70 ms; phần còn lại là thời gian bắt thay đổi và chu kỳ thăm dò của consumer.

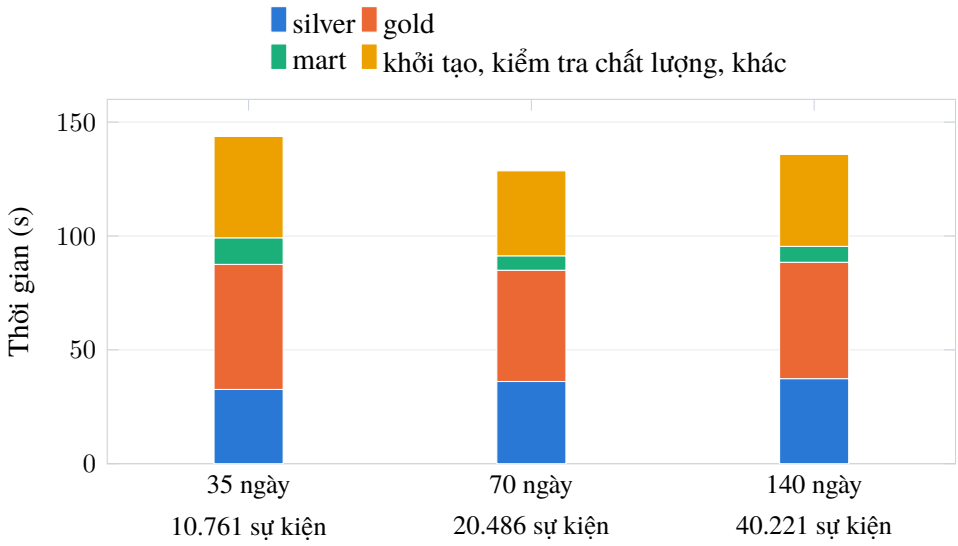
5.4.6 BR7/BR8: lớp phủ thời gian thực

Bảng 27. Độ trễ từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước)

Phiên mỗi đợt	Sự kiện thô (trung bình)	Trung bình (nhỏ nhất – lớn nhất)	Chính xác
25	234	7,27 s (7,19 – 7,35)	có
100	888	7,23 s (6,95 – 7,38)	có
400	3.622	7,12 s (5,69 – 8,03)	có

Độ trễ gần như không đổi khi tải tăng 15 lần, vì nó bị chi phối bởi chi phí cố định của mỗi micro-batch (bước chuẩn hoá khoảng 2,7 s, trigger 1 s) chứ không phải khối lượng. “Chính xác” nghĩa là lớp phủ chứa đúng số tương tác đúng hạn kỳ vọng của đợt cho video nhiều tương tác nhất.

5.4.7 Thời gian build kho dữ liệu



Hình 24. Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần)

Tổng thời gian build là 143,7, 128,6 và 135,8 giây cho 35, 70 và 140 ngày dữ liệu; mọi model đều thành công. Thời gian gần như phẳng khi khối lượng tăng 4 lần: ở quy mô này, chi phí cố định của mỗi model (khởi tạo job Spark, commit Delta, kiểm tra chất lượng) chiếm phần lớn. Các mart BR6–BR8 chỉ mất 1–6 giây. Chênh lệch giữa hai lần chạy (123 và 164 giây ở 35 ngày) là nhiễu do máy ảo dùng chung với phần còn lại của stack.

5.5 Hiệu quả cảnh báo BR5 trên OULAD

Dữ liệu công khai OULAD [2, 24] có 10.655.280 dòng tương tác tổng hợp theo ngày trong lần chạy này. Phép đánh giá tạo 77.106 quan sát tại các ngày 14, 28 và 56. Đặc trưng có trước mốc dự báo, nhãn là rút học trong 30 ngày kế tiếp. Nhóm tách học viên 2013 bằng băm để hiệu chỉnh và xác thực. Tập 2014 loại học viên đã xuất hiện năm 2013 và được trình bày như đánh giá hồi cứu vì đã được xem trong các lượt trước.

Ngưỡng 65 hòa ngưỡng 60 trên tập xác thực 2013 với F1 khoảng 0,128. Nhóm giữ policy v1 ở ngưỡng 60. PR-AUC trong bảng là average precision, còn thời gian báo trước là trung vị của dương tính đúng. Baseline không hoạt động bảy ngày báo sớm hơn ở tập xác thực, với F1 thấp hơn. Cố vấn cần dùng lời giải thích và lịch sử để kiểm tra các cảnh báo vì precision còn thấp. Các mốc lặp của một học viên có tương quan. Tổng hợp theo ngày cũng giới hạn khả năng suy luận về hoạt động trong ngày. Kết quả chưa đo hiệu quả can thiệp.

Bảng 28. BR5 trên OULAD, gộp ba mốc quan sát

Tập	Phương án	Precision	Recall	F1	PR-AUC	Báo trước (ngày)
2013	Policy v1	0,095	0,193	0,128	0,060	10,0
2013	Ngưỡng 65	0,095	0,193	0,128	0,060	10,0
2013	Không hoạt động 7 ngày	0,059	0,344	0,100	0,042	13,0
2014	Policy v1	0,121	0,111	0,116	0,058	11,0
2014	Ngưỡng 65	0,121	0,111	0,116	0,058	11,0
2014	Không hoạt động 7 ngày	0,061	0,350	0,104	0,045	12,5

5.6 Thảo luận

Kết quả BR4 cho phép giảng viên ưu tiên kiểm tra test lỗi và xác minh việc sửa điểm qua publication mới. Với BR5, cố vấn thấy các tín hiệu đóng góp, trạng thái xác nhận, ghi chú và phục hồi trên cùng cảnh báo. Độ trễ ở đây phụ thuộc chu kỳ publish, thời gian truy vấn và tài nguyên của máy đo. Lựa chọn engine cần được đánh giá lại khi thay đổi tải hoặc cấu hình. Các minh chứng giao diện ở Mục 4.2 nối kết quả kỹ thuật với thao tác nghiệp vụ.

6 Hạn chế, hướng phát triển và kết luận

6.1 Hạn chế và hướng phát triển

6.1.1 Hạn chế hiện tại

1. **BR1–BR3 vẫn chạy trong runtime demo.** BR1 và BR3 có dịch vụ Kafka riêng. Giao diện thông báo còn đọc dữ liệu demo. Code Lab Classroom dùng bộ chấm demo và lưu kết quả vào PostgreSQL để cấp cho CDC và BR4. Đường từ giao diện qua worker Kafka riêng cần nghiệm thu thêm.
2. **BR2 chưa có nhánh đặc trưng phiên và mô hình học máy.** Đề bài yêu cầu kết hợp đặc trưng lô và đặc trưng phiên qua feature store. Hiện mới có bảng đặc trưng lô trong lakehouse; việc đẩy bảng này lên Redis và nhánh streaming chưa được hiện thực.
3. **Độ trễ streaming ở mức giây.** Độ trễ quan sát gồm chu kỳ publish và polling. So sánh Spark/Flink phản ánh cấu hình thí nghiệm ở Mục 5.
4. **Embedding mang tính từ vựng.** Bộ nhúng băm không hiểu ngữ nghĩa (“join” không khớp “merge”). Đã có cổng `Embedder` và đường reindex để thay bằng mô hình thật.
5. **Lớp phủ không mang phiên bản nội dung.** Vào ngày một video được quay lại, lớp phủ trộn hai phiên bản; vì vậy nó chỉ được hiển thị cho phiên bản mới nhất.

6. **Vận hành lịch sử BR5.** Điểm và chuyển trạng thái được phát qua outbox. Phép thử merge xác nhận hai lượt cùng điểm nhưng khác policy vẫn được lưu riêng. Cần theo dõi độ trễ đường đồng bộ kho khi vận hành dài hạn.
7. **Hiệu năng BR4.** Mỗi revision làm mới cửa sổ bị ảnh hưởng. Đường Spark ghi thêm facts và dimensions bất biến, nên chi phí công bố khác phép đo tổng hợp đọc JDBC.
8. **Vận hành đơn nút, bảo mật cục bộ.** Trino, OpenSearch, Kafka chạy một nút; plugin bảo mật của OpenSearch tắt; phiên staff sống trong một tiến trình API, hết hạn sau tám giờ hoặc khi khởi động lại. Đã có chẩn đoán offsets và độ tươi, chưa có SLA vận hành nhiều nút.
9. **Độ khái quát.** Bài đo hiệu năng dùng fixture tổng hợp. OULAD cho phép đánh giá dự đoán rút học ngoại tuyến từ tổng hợp theo ngày. Tập 2014 là hồi cứu và chưa có thử nghiệm tác động của can thiệp.
10. **Chỉ số BR7.** Mart đếm thao tác *seek* theo giây và phiên bản. Cần tách tua lùi khỏi tua tới để đáp ứng đầy đủ chỉ số *rewind* trong đề tài.
11. **Lệch dữ liệu mẫu.** Seed của lakehouse khai báo khoảng 18 khoá học và video không có trong cơ sở dữ liệu LMS mẫu, nên job reindex báo các mục này là lỗi.

6.1.2 Hướng phát triển

- Tích hợp API và giao diện với các dịch vụ Kafka hiện có của BR1 và BR3, rồi kiểm chứng đầu cuối.
- Bổ sung nhánh đặc trưng phiên cho BR2 (cửa sổ phiên 30 phút), đẩy `features.learner_longterm` lên Redis, và thử một mô hình gợi ý học từ dữ liệu, so sánh với luật hiện tại.
- Đo thêm dưới tải đồng thời và ngân sách tài nguyên khác trước khi chọn cấu hình hoặc engine cho môi trường lớn.
- Thay bộ nhúng băm bằng mô hình sentence-embedding, chạy một lần reindex bắt buộc.
- Chuyển các bảng snapshot (`fct_enrollment`, `fct_lesson_progress`) sang MERGE dựa trên Change Data Feed khi khối lượng đủ lớn; với dashboard dưới giây ở quy mô lớn, gắn StarRocks vào cùng Hive Metastore.
- Triển khai lên Kubernetes với nhiều nút, TLS và xác thực tập trung; giám sát bằng Prometheus cùng Grafana; bổ sung quy trình xoay khoá bút danh.
- Thu thập nhãn kết quả thật để hiệu chỉnh và đánh giá điểm rủi ro BR5.

6.2 Kết luận

Báo cáo đã trình bày DE_261, một nền tảng kỹ thuật dữ liệu biến bốn nguồn dữ liệu rời rạc của một hệ thống giáo dục trực tuyến thành các kho dữ liệu phục vụ tám yêu cầu nghiệp vụ.

Về kiến trúc, nền tảng bám sát kiến trúc tham chiếu của đề bài: Kafka và Debezium cho tiếp nhận, Spark cho xử lý, Delta Lake cho lakehouse, Redis cho phục vụ độ trễ thấp. Lớp gold được mô hình hoá thành kho dữ liệu Kimball và phục vụ qua Trino. Các điểm điều chỉnh, quan trọng nhất là dùng Spark Structured Streaming thay Flink cho BR5 và các lớp phủ, đều được ghi lại bằng ADR kèm lý do và hệ quả. Tổ chức mã theo DDD và kiến trúc lục giác cho phép cùng một logic nghiệp vụ chạy trên nhiều engine và được kiểm thử độc lập.

BR4 kiểm chứng việc sửa điểm qua CDC và công bố có phiên bản. BR5 kiểm chứng cửa sổ có trạng thái, cảnh báo và phục hồi sau phát lại checkpoint. BR6–BR8 có minh chứng phục vụ kho, heatmap và đồng bộ học liệu. BR1–BR3 còn các bước tích hợp được liệt kê ở Mục 6.1. Giao diện mới đưa dữ liệu đến từng vai trò qua phiên đăng nhập có phạm vi.

Các phép đo tách thời gian tổng hợp, thời gian từ nguồn đến API và hiệu quả sử dụng. PostgreSQL nhanh hơn Spark trong phép đo tổng hợp BR4 đã lưu, trong khi đường Spark/Delta lưu được snapshot, facts và dimensions theo từng lần công bố. Ở cấu hình BR5 đã đo, cả Spark và Flink giữ đúng trạng thái khi khởi động lại. Đánh giá OULAD cho thấy policy nhiều tín hiệu đạt F1 cao hơn baseline không hoạt động bảy ngày trên tập xác thực, với precision tuyệt đối còn thấp. Nhóm giữ policy v1 và dùng lời giải thích điểm để hỗ trợ cố vấn kiểm tra từng trường hợp.

Tài liệu tham khảo

[1] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[2] Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. Open university learning analytics dataset. *Scientific Data*, 4:170171, 2017.

[3] Advanced Distributed Learning Initiative. Experience API (xAPI) Specification. `https://github.com/adlnet/xAPI-Spec`, 2017.

[4] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*, 2011.

[5] Debezium Community. Debezium: Stream changes from your database. `https://debezium.io/documentation/`, 2026. Truy cập 11/2026.

[6] Matei Zaharia, Reynold S. Xin, Patrick Wendell, et al. Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11):56–65, 2016.

[7] Michael Armbrust, Tathagata Das, Joseph Torres, et al. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[8] Michael Armbrust, Tathagata Das, Liwen Sun, et al. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020.

[9] Raghav Sethi, Martin Traverso, Dain Sundstrom, et al. Presto: SQL on Everything. In *IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1802–1813, 2019.

[10] Redis Ltd. Redis Documentation. `https://redis.io/docs/`, 2026. Truy cập 11/2026.

[11] OpenSearch Project. OpenSearch Documentation: k-NN and hybrid search. `https://opensearch.org/docs/latest/`, 2026. Truy cập 11/2026.

[12] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[13] Alistair Cockburn. Hexagonal Architecture (Ports and Adapters). `https://alistair.cockburn.us/hexagonal-architecture/`, 2005.

[14] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[15] Michael Nygard. Documenting Architecture Decisions. `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions`, 2011.

[16] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and Batch Processing in a Single Engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 38(4), 2015.

[17] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.

[18] Databricks. What is the medallion lakehouse architecture? `https://docs.databricks.com/en/lakehouse/medallion.html`, 2026. Truy cập 11/2026.

[19] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.

[20] Latanya Sweeney. k-Anonymity: A Model for Protecting Privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.

[21] Yu. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[22] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.

[23] Hynek Schlawack. How to hash a password. Tài liệu argon2-cffi. `https://argon2-cffi.readthedocs.io/en/stable/howto.html`.

[24] Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. Open university learning analytics dataset. UCI Machine Learning Repository, CC BY 4.0, 2015.

A Hướng dẫn chạy và tái lập kết quả

A.1 Yêu cầu

Docker (khuyến nghị 8 GB RAM cho VM), Python 3.11 trở lên, Node.js 20 trở lên. Postgres của dự án mặc định ở cổng 5433 để tránh xung đột với các dự án khác.

A.2 Khởi động

```
# BR4/BR5: CDC ậtht + Spark Structured Streaming
docker compose --profile br45 up -d --build demo br45-init br45-cdc br45-
    stream
docker compose exec -T br45-cdc python -m edu_platform.br45.cli seed

# BR6/BR7/BR8: kho ữd ệliu, đồng ộb ộni dung, ólp ủph ựtrc ếtip
docker compose --profile br678 up -d --build     # ầln đầu ảkhong 4-5 phút
docker compose exec demo edu telemetry simulate --sessions 50

# Kho ữd ệliu óvi ữd ệliu ổtng ợhp
docker compose run --rm spark-jobs edu warehouse bootstrap
docker compose run --rm spark-jobs edu warehouse seed-bronze --end-date
    2026-10-17 --days 35
docker compose run --rm spark-jobs edu warehouse run --run-date 2026-10-17 --
    start-date 2026-08-01
docker exec -it de261-trino trino --user analyst --catalog lakehouse
```

Mở ứng dụng tại `http://localhost:8000`. Tạo tài khoản Instructor, Advisor và Admin bằng lệnh `python -m edu_platform.br45.auth`, rồi truyền JSON qua `EDU_STAFF_ACCOUNTS`. Hướng dẫn đầy đủ nằm ở `docs/STAFF_AUTH.md`. Dùng `docker-compose.br45-classroom.yml` cho Student có phiên riêng và `docker-compose.br45-replay.yml` cho kịch bản mô phỏng. Mỗi chế độ cần ghi rõ khi chụp minh chứng.

A.3 Kiểm thử

```
.venv/bin/ruff check src tests && .venv/bin/mypy
.venv/bin/python -m pytest tests/unit
.venv/bin/python -m pytest -m spark tests/integration       # ầcn JDK
# Set BR45_VERIFY_INSTRUCTOR_TOKEN and BR45_VERIFY_ADVISOR_TOKEN.
EDU_BR45_E2E=1 .venv/bin/python -m pytest tests/e2e/test_br45.py
cd apps/web && npm ci && npm run build && npx playwright install chromium
```

```
# Configure the staff test credentials per docs/STAFF_AUTH.md.
BR45_BASE_URL=http://localhost:8000 npx playwright test
```

A.4 Đo hiệu năng

```
bash infra/br45/run-engine-benchmark.sh prepare
bash infra/br45/run-engine-benchmark.sh br4 report-run
bash infra/br45/run-engine-benchmark.sh spark report-run
bash infra/br45/run-engine-benchmark.sh flink report-run
```

Chạy từng engine trên hạ tầng đối chiếu cô lập. Nhật ký tháng 11, kết quả oracle và bảng OULAD nằm trong `data/br45-restoration/`. CSV tháng 10 và số liệu BR6–BR8 nằm trong `data/`. Để đo worker triển khai, dùng hai project Compose và đặt `EDU_BR45_ASSESSMENT_ENGINE` lần lượt là `postgres` và `spark`. Cố định giờ UTC, nạp fixture 24 học viên, đo qua API đã cấp token. Script `data/measure_deployed_br4.py` ghi từng lượt, gồm mốc commit, số đếm, revision và định danh bản công bố.

Đánh giá OULAD chạy từ gốc repo. Tệp ZIP tải từ UCI [24] phải có SHA-256 ghi trong `oulad_evaluation.json`.

```
python -m edu_platform.br45.effectiveness \
  --archive "$OULAD_ARCHIVE" --output "$OULAD_RESULTS"
```

A.5 Biên dịch báo cáo

```
cd "DE_Latex_Report/New Project"
tectonic --reruns 4 main.tex # ặhoc: latexmk -xelatex main.tex
```

Trên Overleaf, chọn trình biên dịch XeLaTeX.

B Tóm tắt API

Tất cả endpoint có tiền tố `/api`. Tài liệu OpenAPI đầy đủ ở `http://localhost:8000/docs`. Hợp đồng chi tiết ở `docs/api.md`. API staff dùng cookie phiên kèm `X-Staff-Role` hoặc Bearer token đã khai báo. Phiên thiếu hay hết hạn trả 401. Sai vai trò, sai origin khi ghi bằng cookie hoặc ngoài phạm vi trả 403. Dữ liệu không có trả 404, truy vấn sai trả 422, vượt giới hạn thử mật khẩu trả 429 và kho chưa sẵn sàng trả 503.

Bảng 29. Các endpoint chính

BR	Phương thức	Đường dẫn	Mục đích
Auth	POST, GET DELETE	`/auth/staff/{role}/session`	Đăng nhập, lấy danh tính và thu hồi phiên
Auth	POST, GET DELETE	`/classroom/student/session`	Phiên Student trong Classroom
Auth	POST	`/classroom/student/lessons/{id}/activity`	Lưu hoạt động bài học của Student
1	GET	`/learners/{id}/notifications`	Thông báo của người học
2	GET	`/learners/{id}/recommendations`	Gợi ý và khoá đã ghi danh
3	POST	`/submissions`	Nộp bài (202); Classroom lưu phản hồi và kết quả test
3	GET	`/submissions/{id}`	Kết quả và phản hồi
4	GET	`/instructor/courses/{course}/questions/statistics`	Thống kê câu hỏi
4	POST	`/instructor/courses/{course}/assessment-jobs`	Yêu cầu công bố theo cửa sổ
4	GET	`/instructor/assessment-jobs/{id}`	Trạng thái, source revision và Delta version
4	GET	`/instructor/questions/{question}/test-cases`	Thống kê test case
5	GET	`/advisor/courses/{course}/at-risk-students`	Hàng đợi rủi ro, tóm tắt mức
5	GET	`/advisor/students/{student}/risk-history`	Lịch sử điểm
5	GET	`/advisor/alerts`	Danh sách cảnh báo
5	PATCH	`/advisor/alerts/{id}/acknowledge, /resolve`	Vòng đời cảnh báo
5	GET, PUT	`/advisor/courses/{course}/policy`	Phiên bản policy và mốc hiệu lực
5	GET, POST	`/advisor/alerts/{id}/notes`	Ghi chú có idempotency key

BR	Phương thức	Đường dẫn	Mục đích
6	GET	`/admin/organisations/{org}/skill-progress`	Báo cáo tổ chức (k = 5)
7	GET	`/instructor/courses/{course}/videos`	Video và phiên bản
7	GET	`/instructor/videos/{id}/interaction-heatmap`	Heatmap kèm lớp phủ
8	GET	`/instructor/courses/{course}/engagement-trend`	Xu hướng tương tác
8	POST	`/instructor/content/{id}/versions`	Xuất bản phiên bản (202)
8	GET	`/instructor/courses/{course}/search`	Tìm kiếm lai hoặc từ khoá
–	GET	`/events`	Server-Sent Events cho sự kiện trực tiếp